ESCOLA POLITÉCNICA DA UNIVERSIDADE DE SÃO PAULO
Departamento de Engenharia de Computação e Sistemas Digitais

PCS3635 – LABORATÓRIO DE PROJETO DE SISTEMAS DIGITAIS I

PROJETO SomLetrando – Semana 2
Relato da Bancada B8 – Turma 2 – Prof. Antonio Vieira

Data de Emissão: 27 de março de 2026.

Nome: Enzo Pimentel Lorenzon	**Número USP: 14568409**
Nome: Altair Raphael Alcazar Perez	**Número USP: 14555666**
Nome: Giovana Une Oyakawa	**Número USP: 15455343**

Sumário

1 Introdução **5**

2 Especificação Técnica do Projeto **6**

2.1 Contexto de Aplicação do Projeto **6**

2.2 Objetivos do Projeto **6**

2.3 Descrição de Recursos e Custos **6**

2.4 Definição da Arquitetura do Sistema **7**

2.4.1 Descrição da Arquitetura Comportamental 7

2.4.2 Storytelling da Interação entre Usuários e Sistema *7*

2.4.3 Descrição da Arquitetura Estrutural 8

2.5 Especificação de Requisitos **8**

2.6 Cronograma do Projeto **9**

3 Semana 1 de Desenvolvimento do Projeto **10**

3.1 Descrição Funcional do Projeto para a Semana **10**

3.2 Detalhamento do Projeto Lógico do Hardware Programável **10**

3.2.1 Projeto do Fluxo de Dados 10

3.2.2 Projeto da Unidade de Controle 10

3.2.3 Projeto do Sistema Digital Programável 11

3.3 Plano de Testes do Hardware Programável e Simulações **11**

3.3.1 Cenário de Teste 1 – Aleatorização da ROM 11

3.3.2 Cenário de Teste 2 – Validação do botão “confirmar” 11

3.4 Detalhamento do Projeto de Software **12**

3.4.1 Descrição da Arquitetura de Software 12

3.4.2 Detalhamento do Projeto de Software 12

3.5 Plano de Testes do Software **12**

3.5.1 Cenário de Teste X – <Objetivo Sucinto do Cenário> 12

3.6 Integração entre Hardware Programável e Software **13**

3.7 Prototipação do Projeto **13**

3.8 Implantação do Projeto **14**

3.8.1 Pinagem da Placa FPGA 14

3.8.2 Estratégia de Montagem 14

3.8.3 Estratégia de Depuração 14

3.8.4 Execução Prática do Cenário de Teste 1 – Aleatorização da ROM 15

3.8.5 Execução Prática do Cenário de Teste 2 – Validação do botão “confirmar” 16

3.9 Conclusão da Semana **16**

4 Semana 2 de Desenvolvimento do Projeto **17**

4.1 Descrição Funcional do Projeto para a Semana **17**

4.2 Detalhamento do Projeto Lógico do Hardware Programável **17**

4.2.1 Projeto do Fluxo de Dados 17

4.2.2 Projeto da Unidade de Controle 17

4.2.3 Projeto do Sistema Digital Programável 18

4.3 Plano de Testes do Hardware Programável e Simulações **18**

4.3.1 Cenário de Teste X – <Objetivo Sucinto do Cenário> 18

4.4 Detalhamento do Projeto de Software **19**
4.4.1 Descrição da Arquitetura de Software 19
4.4.2 Detalhamento do Projeto de Software 19
4.5 Plano de Testes do Software **19**
4.5.1 Cenário de Teste X – <Objetivo Sucinto do Cenário> 19
4.6 Integração entre Hardware Programável e Software **20**
4.7 Prototipação do Projeto **20**
4.8 Implantação do Projeto **21**
4.8.1 Pinagem da Placa FPGA 21
4.8.2 Estratégia de Montagem 21
4.8.3 Estratégia de Depuração 21
4.8.4 Execução Prática do Cenário de Teste X – <Objetivo Sucinto do Cenário> 22
4.9 Conclusão da Semana **23**
5 Conclusões do Projeto **24**
6 Referências Bibliográficas **25**

1 INTRODUÇÃO

Como tema de projeto da disciplina PCS3635 - Laboratório de Projetos de Sistemas Digitais, foi proposto a criação de soluções voltadas para a comunidade TEA (Transtorno do Espectro Autista). Inicialmente foram feitos trabalhos de pesquisa para entender melhor os desafios enfrentados e as necessidades do público-alvo. Com isso, foi possível definir escopo para o projeto: Auxílio na criação de consciência fonológico de indivíduos no espectro. Historicamente, apenas cerca de 50% de pessoas no espectro TEA alcançam um nível de domínio de linguagem suficiente para interagir com o meio externo com menores impactos negativos, os outros 50%, não adquirindo essa habilidade, não raramente acabam recorrendo a métodos não convencionais de comunicação (Ex.: autoflagelação, destruição de propriedade, agressividade no geral, etc.) [1].

Para mitigar esses efeitos negativos, e melhorar a qualidade de vida tanto dos indivíduos no espectro, quanto das pessoas no seu entorno, é utilizada a abordagem fonológica durante a alfabetização. Na verdade, essa é uma metodologia de ensino eficiente tanto para neurotípicos, quanto atípicos[2]. Entretanto, para abranger, também, alunos não verbais, é necessário o uso de uma abordagem conhecida como Nonverbal Reading Approach (NRA)[3]. Ela, com o auxílio de sight words[4], permite que seja utilizada a voz interna do indivíduo (a mesma que ocorre ao ler textos em silêncio) ao invés de verbalizar as palavras oralmente.

Tendo isso em mente, será desenvolvido um jogo de nome comercial somLetrando, com o objetivo de ensinar e praticar a associação de sons com letras para formar palavras de maneira lúdica, auxiliando no desenvolvimento da consciência fonológica. Um alto-falante externo será responsável por reproduzir o som de uma palavra qualquer contida na memória do circuito e o usuário terá à disposição um painel contendo botões de diversas cores, cada uma associada ao som de uma letra. Com isso, vence o jogo se o jogador for capaz de selecionar na ordem correta as letras que compõem a palavra escolhida a partir de seus sons.

2 Especificação Técnica do Projeto

O objetivo desta seção do documento é detalhar a especificação técnica do projeto SomLetrando.

2.1 Contexto de Aplicação do Projeto

O espectro TEA abarca 3 níveis de suporte: nível 1 (autismo leve), nível 2 (autismo moderado) e o nível 3 (autismo severo). Em relação ao ensino, o primeiro nível possui maior facilidade de aprendizado quando há inclusão de reforços visuais, já o segundo nível tem maiores dificuldades de comunicação tanto verbal quanto não verbal, por último o terceiro nível demanda uma maior dedicação nas ferramentas de aprendizagem como uso de comunicação alternativa e sistemas multissensoriais.

Para crianças com TEA, a previsibilidade, a rotina e os estímulos sensoriais adequados são elementos fundamentais para a consolidação da aprendizagem na medida em que essas características reduzem a ansiedade e aumentam o foco [5].

Nesse sentido, o projeto se encontra bem definido, integrando múltiplos recursos sensoriais visuais (interfaces gráficas, leds), sonoros (alto falante reproduzindo letras) e táteis (personalização de textura dos botões) e estimulando a aprendizagem de todos os níveis do espectro TEA, contidos na faixa etária de 2 a 12 anos. Esse grupo foi escolhido por existir uma gama de estudos voltadas a esse público mais extensa, infelizmente, o impacto que tais medidas podem ter na educação de pessoas mais velhas ainda não foi estudado com o mesmo empenho. Portanto, é possível ter maior confiabilidade do impacto dos métodos utilizados nesse projeto para o público infantil. Entretanto, nada garante que, com o acompanhamento adequado de profissionais, esse jogo não possa trazer um aumento da qualidade de vida de pessoas fora da faixa etária alvo.

Contudo, vale ressaltar que o aprendizado de autistas, mesmo aqueles que pertencem a um mesmo nível, é completamente distinto entre si. Por conta disso, a mediação social através do professor, algum profissional ou pais é extremamente relevante para facilitar a decodificação simbólica e a formação de memórias linguísticas duradouras.

Assim, o projeto constitui-se como um objeto relevante até mesmo como ferramenta de uso em sala de aula pelos educadores ou em casa pelos pais, não só mediando o aprendizado como também construindo a confiança da criança e fortalecendo

vínculos emocionais.

2.2 Objetivos do Projeto

Dado o contexto de aplicação do projeto para pessoas com TEA, os objetivos podem ser resumidos em:

- Auxiliar o desenvolvimento da comunicação das crianças com TEA, por meio da criação da consciência fonológica;
- Configurar adequadamente os 3 componentes principais do projeto: Placa FPGA, painel de botões e computador externo;
- Criar uma interface de usuário interativa, intuitiva e confortável, para que o foco seja única e exclusivamente o aprendizado dos fonemas e não as regras do jogo. Essa interface será dependente de um computador externo;
- Captar a atenção do público-alvo;
- Desenvolver um banco de palavras para o jogo;

Para atingir estes objetivos de forma eficaz, o projeto fará uso de recursos técnicos específicos, pensados para mitigar as barreiras de aprendizagem:
Configurar adequadamente os 3 componentes principais do projeto: Placa FPGA, painel de botões e computador externo;

- A substituição de teclados convencionais por botões do tipo arcade, criando um painel tangível, reduzirá a sobrecarga motora e sensorial. Isso possibilitará captar a atenção de crianças com desafios de motricidade fina.
- O processamento central na placa DE0-CV garantirá o controle do tempo entre cliques, recurso que é muito importante para evitar que toques acidentais registrem entradas duplicadas.
- Através da comunicação serial, o computador externo reproduzirá os fonemas armazenados no banco de palavras. Tal reprodução deve ter sincronia de forma que prenda a atenção do usuário.

Com esses objetivos e métodos, espera-se que o jogo desenvolvido seja de interesse não apenas para a disciplina e o desenvolvimento da ciência na Universidade, como também cumpra o caráter de atividade extensionista auxiliando na melhor integração de pessoas com o espectro TEA na sociedade.

2.3 Descrição de Recursos e Custos

a) Custos e Recursos

Os custos planejados inicialmente envolvidos no projeto estão resumidos na tabela abaixo. Essa tabela poderá variar conforme a evolução do projeto e necessidades verificadas pelo grupo.

Tabela 1 – Lista de Materiais e Custo do Projeto

Item	Quantidade	Custo Unitário	Custo Total R$	Referência
Placa FPGA DE0-CV	1	0	0	Usada do laboratório
Botão para Fliperama Arcade Push Button PBS-29	10	7,70	77,00	Saravati
Chave Gangorra Redonda KCD1-106N	5	3,20	16	Saravati
Leds	5	1,00	5,00	Saravati
Kit resistores 10kOhm	3	0,71	2,13	Robocore
Resistores 330 Ohms	3	0,80	2,40	Eletrogate
Módulo Serial CP2102	1	27,00	27,00	Professor Midorikawa
Computador pessoal	1	0	0	Alunos
Impressão do manual de usuário (3pág cada)	3	2,00/pág	20,00	Copiadoras
Caixa organizadora	1	17,90	17,90	Saravati
1 kit de 40	1	7,50	7,50	Robocore

Item	Quantidade	Custo Unitário	Custo Total R$	Referência
jumpers				
Filamento de Impressão (protótipo caixa)	1	37,48	37,48	3D Lab
Retalhos com texturas diversas	8	5,00	40,00	Lojas físicas (brás, 25, etc)
Custo Total do Projeto:			252,41	

Após pesquisa da semana 0.2 e a conversa com o Monitor Gabriel, foram adicionados para essa especificação completa: retalhos de tecidos com texturas que irão ser colocadas por cima dos botões para auxiliar no desenvolvimento tátil dos autistas uma margem de 10% na quantidade de componentes comprados visando prever quaisquer tipos de falhas ou erros de execução do projeto. Além disso, com a implementação de dois modos de jogo, foi adicionada mais uma chave ao projeto, além de um LED que sinaliza vitória para estimular a sinalização positiva.

b) Quantidade de horas de trabalho semanais planejadas pela equipe

Para o desenvolvimento adequado do projeto dentro do tempo estipulado, o grupo pretende dedicar, além do tempo de aula, um total de 5h para cada integrante, distribuídas ao longo da semana. Sabe-se que períodos com maior necessidade de desenvolvimento, tais como a semana 4 e 5 (*openlab)*, exigirão uma carga horária de dedicação maior caso ocorram complicações durante a elaboração do projeto.

2.4 Definição da Arquitetura do Sistema

Nesta seção, são apresentadas as descrições estrutural e comportamental do sistema correspondente ao projeto SomLetrando. Essa descrição é dividida em três partes: (i.) comportamental, com a dinâmica de funcionamento do sistema, (ii.) *storytelling* das interações dos usuários com o sistema e (iii.) estrutural, com a identificação dos blocos funcionais e de suas partes.

2.4.1 Descrição da Arquitetura Comportamental

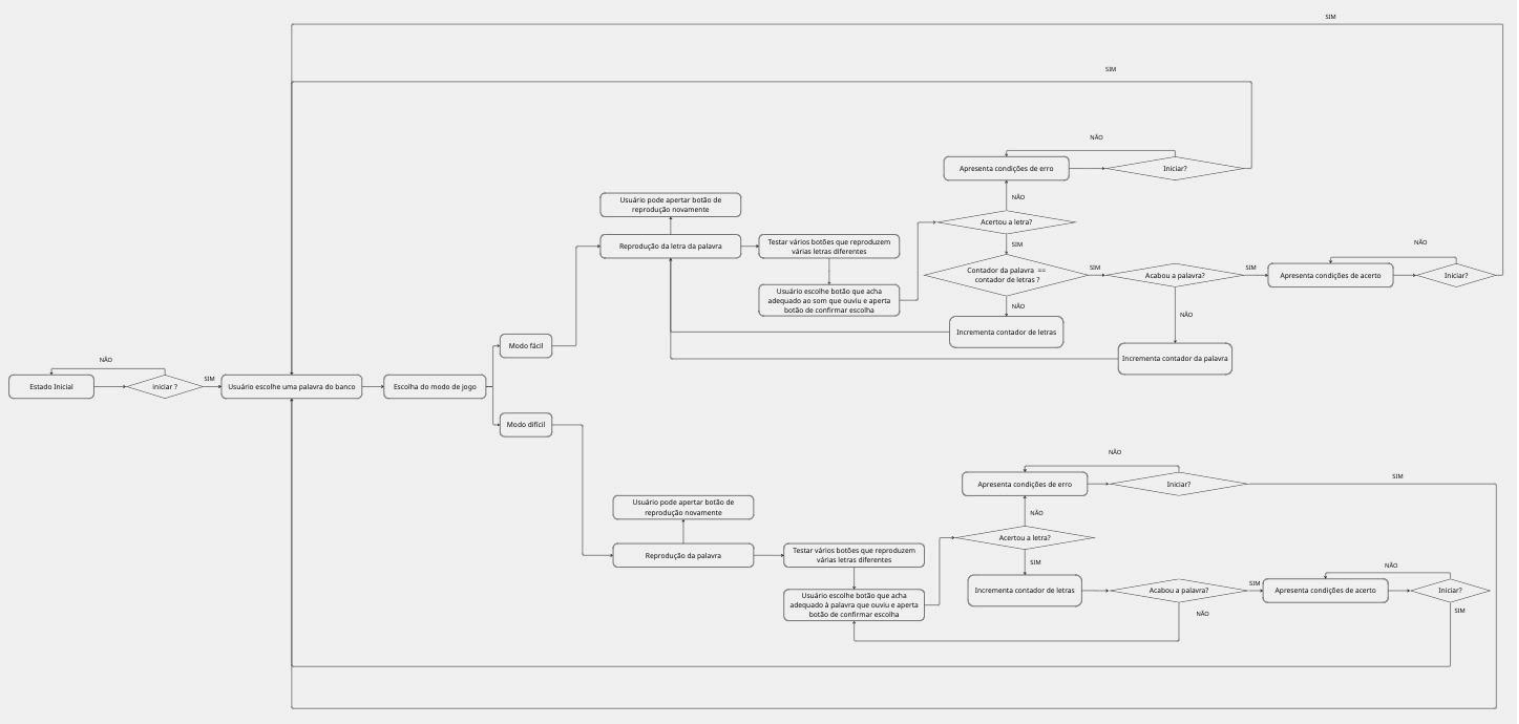

Figura 1: Fluxograma da execução do projeto

Inicialmente, o usuário deve iniciar o dispositivo e depois escolher uma palavra do banco com o modo de jogo (fácil ou difícil). Caso o usuário opte pelo modo fácil, o projeto irá reproduzir a primeira letra da palavra em áudio e permitirá o teste dos botões para que o jogador encontre o botão que presume ser o correspondente ao som que ouviu até que aperte o botão confirma. Caso acerte, o projeto faz com que o usuário tente acertar a primeira letra novamente e depois acerte a segunda letra e assim sucessivamente até completar a palavra. No caso do modo difícil, o usuário ouve a palavra por completo e depois deverá soletrar a palavra por completo, acertando os botões em sequência, mas ainda podendo utilizar os botões de confirmar a escolha e repetir a palavra. Nos dois modos, caso o jogador erre alguma letra em qualquer momento, o projeto vai para um estado de erro e reproduz em tela: “Boa tentativa, não desanime e tente novamente” e reproduz um som de perda. O usuário pode apertar o botão para repetição da letra (modo fácil) ou palavra (modo difícil) a qualquer momento e pode testar múltiplas vezes os sons das letras guardadas nos botões. Ao finalizar o modo de jogo corretamente, o jogo imprime em tela uma frase “Uhul!! Você acertou, vamos jogar mais?”, além de acender um led verde de vitória no painel de botões e reproduzir a frase em alto falante.

2.4.2 Storytelling da Interação entre Usuários e Sistema

O usuário principal, uma criança com TEA, senta-se diante de uma caixa com botões coloridos e uma tela à sua frente. O computador emite o som da palavra “MESA”, e a criança aperta os botões coloridos relacionados aos sons “M”, “E”, “S”, “A”, nesta ordem. A cada acerto, o sistema encoraja a continuação, até emitir uma mensagem final de vitória. No caso de erro, uma mensagem de “Tente Novamente” é exibida. Além dos botões sonoros, duas alavancas, uma para iniciar o jogo, e outra para o nível de dificuldade podem ser selecionadas pelo usuário. Por fim, dois botões auxiliares de repetir escolha e de confirmar escolha se encontram na caixa.

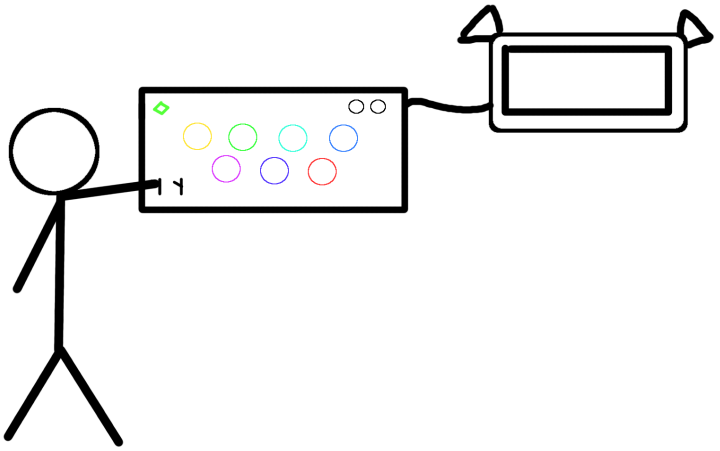

Figura 2: Criança escolhe modo de jogo

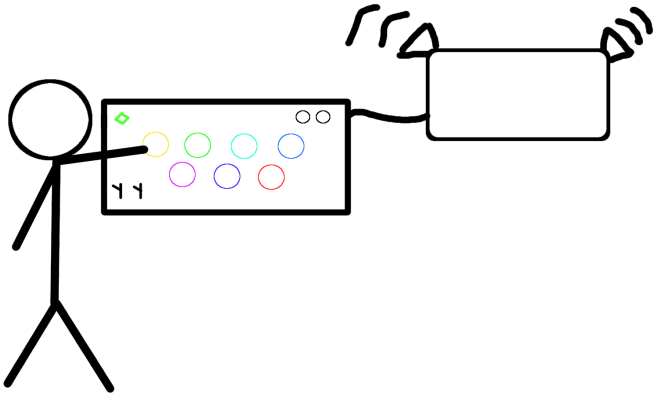

Figura 3: Criança interage com um botão e tem feedback sonoro

2.4.3 Descrição da Arquitetura Estrutural

Tabela 2 – Descrição das Interfaces entre Blocos do Sistema

Código	Descrição da Interface
A	Painel com botões coloridos - Interface de Entrada
B	Placa FPGA DE0-CV - Unidade de Processamento Lógico
C	Software do Computador - Processamento de Áudio
D	Interface gráfica para o usuário a ser exibida no computador

Figura 4: Diagrama de blocos - visão geral

O sistema seguirá uma arquitetura simplista: Entrada de dados do usuário será feita através de botões externos conectados a placa FPGA-DE0CV, ela será responsável por receber os sinais vindos do painel de botões, ler o conteúdo da memória e retransmitir os dados para o computador externo, que os processa e replica os áudios necessários.

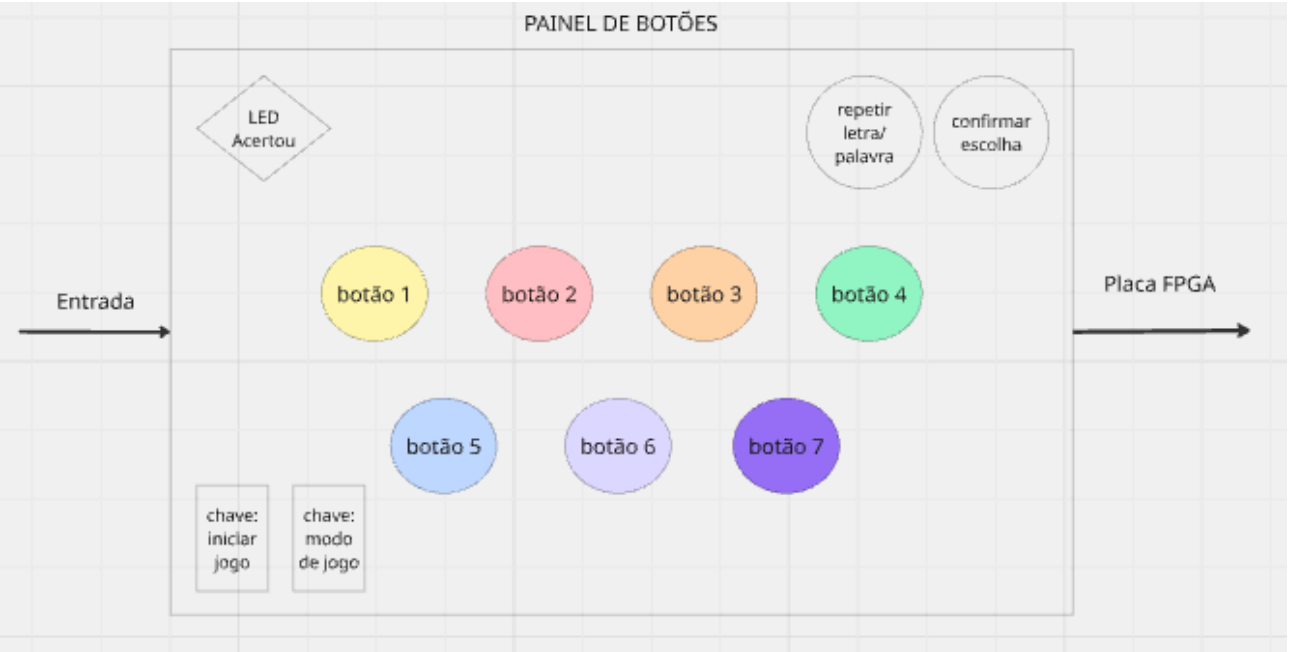

Figura 5: Diagrama do painel de botões (A)

Nesse diagrama, os botões foram coloridos para representação de que possuem texturas distintas. Como o projeto possuirá 7 letras aleatórias a cada jogada, cada uma associada a um botão, então os botões foram nomeados de 1 a 7 (Figura 3). Além disso, também foram representados uma chave para iniciar o jogo, LED's que indicam acerto e erro do usuário, um botão auxiliar caso o usuário deseje, quantas vezes quiser, repetir a palavra objetivo e, por fim, um botão que confirma a escolha do botão com base no último botão apertado.

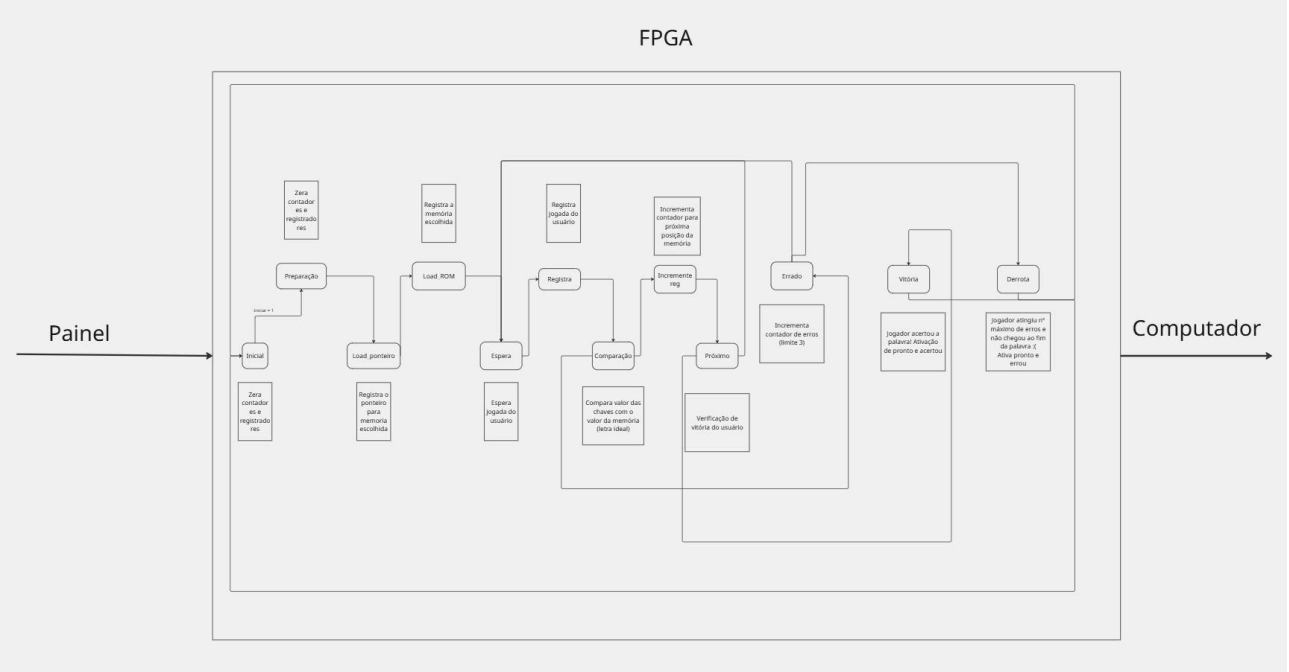

Figura 6: Diagrama da estrutura interna da FPGA (B)

O diagrama de blocos da arquitetura interna da FPGA ilustra integração de seus dois componentes centrais. O núcleo do circuito é coordenado pela Unidade de Controle, que opera a Máquina de Estados Finita para gerenciar as regras e transições do jogo, e pelo Fluxo de Dados, responsável pelo armazenamento estrutural e processamento matemático. O esquema demonstra o roteamento dos sinais físicos de entrada (como *clock*, *reset*, chave de *modo* e os *botões* do usuário) alimentando ambos os blocos . Em contrapartida, as saídas englobam os sinais de status da partida (ganhou, perdeu, pronto) e as vias de depuração, que são mapeadas diretamente para os displays de 7 segmentos e LEDs da placa para monitoramento.

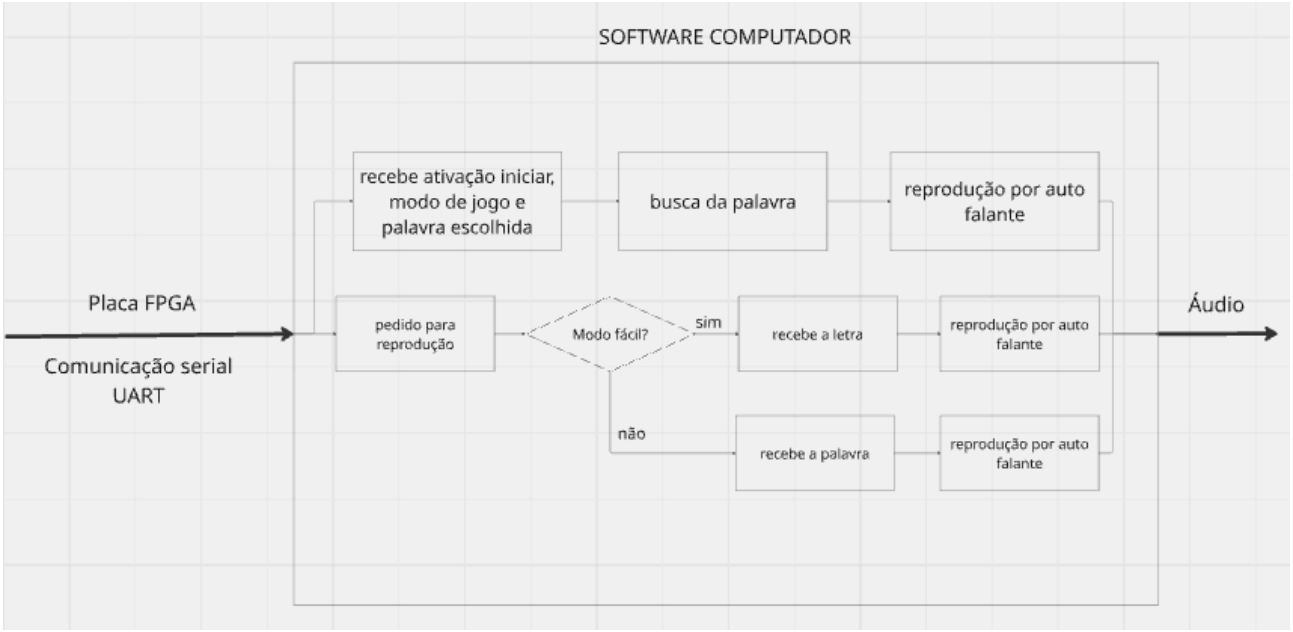

Figura 7: Diagrama de blocos do Software do Computador (C)

A figura acima ilustra o diagrama de blocos do software executado no computador externo, cujo papel é gerenciar o feedback sonoro do SomLetrando. A Interface com o hardware físico é feito por comunicação serial UART, pela qual o software recebe da FPGA os comandos de inicialização, o modo de jogo selecionado e os identificadores da palavra atual. A partir desses dados, o sistema busca o áudio correspondente e bifurca sua lógica de reprodução com base no nível de dificuldade. A saída final de todo esse fluxo é o estímulo de áudio emitido pelo auto-falante.

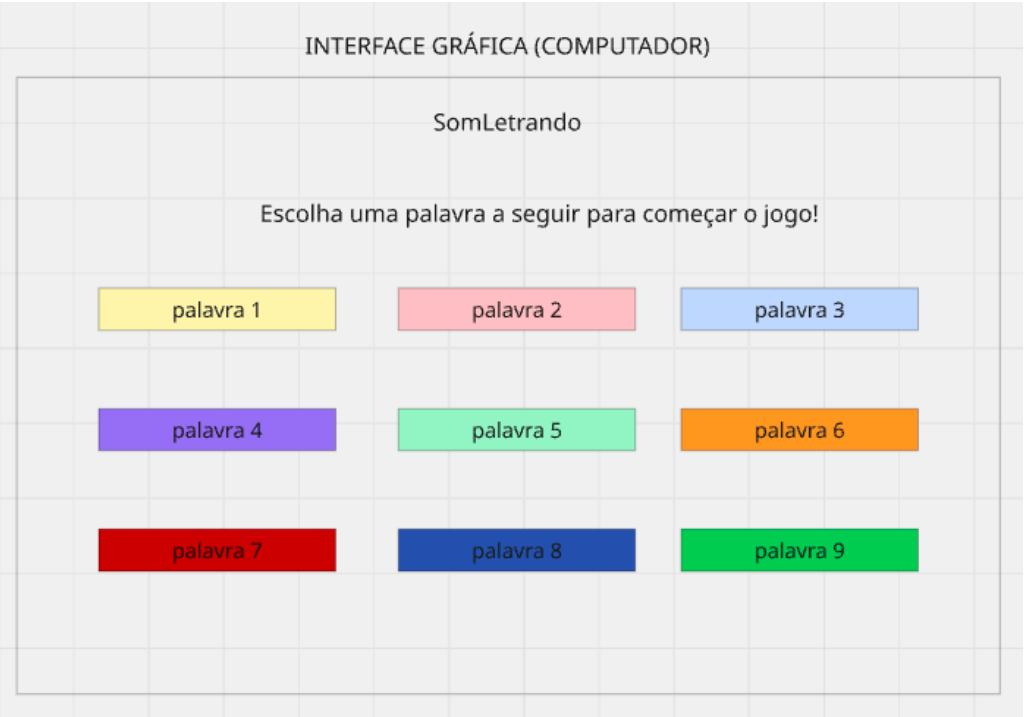

Figura 8: Interface gráfica para o usuário (D)

A interface gráfica apresentada na tela do computador visa ser o mais simples e intuitiva possível, de forma a facilitar o entendimento e manuseio das pessoas no espectro autista. Serão escolhidas 9 palavras para compor o banco de dados das palavras para livre escolha do usuário. Sua tarefa será soletrar a palavra escolhida.

2.5 Especificação de Requisitos

Nesta seção, são apresentados os requisitos funcionais e não funcionais do projeto SomLetrando. Os requisitos são especificados com base na abordagem proposta em [1], que prevê que cada requisito compreende as seguintes informações:

- Código de identificação;
- Tipo (funcional ou não funcional);
- Título;
- Descrição (com frases assertivas e sujeitos ativos);
- Prioridade;
- Estabilidade;
- *Rationale* (informações adicionais e justificativas);
- Requisitos associados.

Tabela 3 – Especificação do Requisito REQ-01

Código: REQ-01	☒Funcional □ Não Funcional
Requisito: Interface Física de Entrada	
Descrição: O sistema deve possuir um painel tangível composto por pelo menos 7 botões do tipo Arcade, onde cada botão será mapeado para um fonema/letra específico.	
Prioridade: ☒ Alta □ Média □ Baixa	
Estabilidade: ☒ Alta □ Média □ Baixa	
***Rationale*:** O uso de botões grandes e coloridos melhora a resposta tátil e reduz a sobrecarga sensorial em comparação com o uso de um teclado convencional de computador.	
Requisitos associados: REQ-02	

Tabela 4 – Especificação do Requisito REQ-02

Código: REQ-02	□Funcional ☒ Não Funcional
Requisito: Sincronia Audiovisual	
Descrição: O Sistema deve garantir a emissão do estímulo sonoro da palavra com uma latência imperceptível .	
Prioridade: ☒ Alta □ Média □Baixa	
Estabilidade: □ Alta ☒ Média □ Baixa	
***Rationale*:** Estímulos sincronizados e previsíveis são essenciais para a fluidez do jogo e para evitar gerar frustração ou perda de foco do usuário	
Requisitos associados: REQ-01	

Tabela 5 – Especificação do Requisito REQ-03

Código: REQ-03	☒Funcional □ Não Funcional
Requisito: Feedback Audiovisual de Validação	
Descrição: O sistema deve fornecer um feedback claro, imediato e previsível para cada acerto ou erro da criança.	
Prioridade: ☒ Alta □ Média □Baixa	
Estabilidade: ☒ Alta □ Média □ Baixa	
***Rationale*:** Crianças com TEA se beneficiam da previsibilidade e de relações claras de causa e efeito. Dessa forma, o feedback imediato reforça o acerto do usuário.	
Requisitos associados: REQ-01, REQ-02.	

Tabela 6 – Especificação do Requisito REQ-04

Código: REQ-04	□ Funcional ☒ Não Funcional
Requisito: Tolerância a toques acidentais estereotipias	
Descrição: O sistema da placa FPGA deve conter uma lógica de debouncing e um tempo de delay mínimo entre leituras sucessivas do mesmo botão, evitando que tremores ou toques múltiplos acidentais registrem a mesma letra/fonema de forma repetida.	
Prioridade: □ Alta ☒ Média □ Baixa	
Estabilidade: ☒ Alta □ Média □ Baixa	
***Rationale*:** Crianças com TEA podem apresentar estereotipias motoras, dessa forma o sistema deve ser tolerante a essas características, não penalizando a criança por toques não intencionais.	
Requisitos associados: REQ-01	

Tabela 7 – Especificação do Requisito REQ-05

Código: REQ-05	☒ Funcional □ Não Funcional
Requisito: Seletor de dificuldade	
Descrição: Permitir que o usuário defina se o sistema deverá reproduzir o som da leitura da palavra ou reproduzir o som de cada letra na sequência adequada	
Prioridade: □ Alta ☒ Média □ Baixa	
Estabilidade: ☒ Alta □ Média □ Baixa	
***Rationale*:** Permite que o usuário possa adequar o nível de dificuldade do jogo para corresponder a suas competências.	
Requisitos associados: -	

Tabela 8 – Especificação do Requisito REQ-06

Código: REQ-06	☒ Funcional □ Não Funcional
Requisito: Coleta e exibição de métricas de desempenho	
Descrição: O sistema deve calcular métricas do desempenho (acertos) do usuário e exibi-las de forma contínua nos displays de 7 segmentos.	
Prioridade: □ Alta □ Média ☒ Baixa	
Estabilidade: □ Alta ☒ Média □ Baixa	
***Rationale*:** Coletar métricas do usuário permite que cuidadores ou terapeutas possam avaliar quantitativamente a evolução fonológica da criança.	

Requisitos associados: REQ-02, REQ-05

Tabela 9 – Especificação do Requisito REQ-07

Código: REQ-07	☒Funcional □ Não Funcional
Requisito: Aleatorização dos botões correspondentes às letras e da escolha de palavras	
Descrição: O sistema deve escolher aleatoriamente a palavra a ser soletrada em cada partida, bem como aleatorizar a atribuição das letras aos botões, ou seja, as letras não terão posições fixas no painel.	
Prioridade: □ Alta □ Média ☒ Baixa	
Estabilidade: ☒ Alta □ Média □ Baixa	
***Rationale*:** Aleatorizar o funcionamento do jogo faz com que o usuário não possa buscar "atalhos" por meio de memorização de padrões de repetição e de fato desenvolva a competência fonológica.	
Requisitos associados: REQ-01, REQ-02	

Tabela 10 – Especificação do Requisito REQ-08

Código: REQ-08	☒Funcional □ Não Funcional
Requisito: Configuração da placa FPGA	
Descrição: O sistema deve ser capaz de escolher e ler o conteúdo de memória, além responder a sinais entrados por meio de botões na própria placa (Iniciar e Reset).	
Prioridade: ☒ Alta □ Média □ Baixa	
Estabilidade: ☒ Alta □ Média □ Baixa	
***Rationale*:** A placa FPGA será a fundação de todo esse sistema, ela será responsável por interpretar os sinais entrados pelo usuário, bem como enviá-los para o computador, que servirá como interface de áudio.	
Requisitos associados: -	

Tabela 11 – Especificação do Requisito REQ-09

Código: REQ-09	☒Funcional □ Não Funcional
Requisito: Estímulo Tátil nos botões	
Descrição: O Painel de botões deve possuir tecidos texturizados distintos acoplados sobre a superfície de cada botão.	
Prioridade: ☒ Alta □ Média □ Baixa	
Estabilidade: ☒ Alta □ Média □ Baixa	
***Rationale*:** O uso de múltiplas texturas auxilia no desenvolvimento tátil de usuários no espectro autista, criando mais uma camada de previsibilidade e reconhecimento multissensorial, além do visual e sonoro.	
Requisitos associados: REQ-01	

Tabela 12 – Especificação do Requisito REQ-10

Código: REQ-10	☒Funcional □ Não Funcional
Requisito: Dinâmica de Fim de jogo e Limite de erros	
Descrição: O sistema deve contabilizar as falhas na formação da palavra. Ao errar uma letra, o sistema deve manter o usuário na mesma posição da palavra. Se o usuário acumular 3 erros na mesma rodada, a partida deve ser encerrada e entrar em estado de derrota.	
Prioridade: ☒ Alta □ Média □ Baixa	
Estabilidade: ☒ Alta □ Média □ Baixa	
***Rationale*:** Limitar as tentativas impõe um desafio lógico razoável à criança, permitindo que a partida seja reiniciada para evitar frustração prolongada.	
Requisitos associados: REQ-03	

2.6 Cronograma do Projeto

O cronograma do projeto **SomLetrando** é apresentado na Tabela 4. Nele, cada um dos requisitos especificados na seção 2.5 está alocado aos marcos das semanas de desenvolvimento do projeto. A presença de um “X” em uma célula da tabela indica que o requisito cujo código de identificação foi definido na seção 2.5 é previsto para ser implementado e demonstrado na(s) respectiva(s) semana(s) de desenvolvimento.

Tabela 13 – Cronograma do Projeto

Requisito	Semana 1	Semana 2	Semana 3	Semana 4
Configuração inicial da FPGA e UART REQ-08		X	X	
Leitura de memória e sinais de controle da FPGA REQ-10	X			
Montagem estrutural do painel físico REQ-01 e REQ-09			X	
Integração do painel com FPGA e testes para duplo clique REQ-04			X	X
Sincronia Audiovisual REQ-02 e REQ-03		X		
Comunicação da FPGA com o PC REQ-02 e REQ-03		X	X	
Lógica da Máquina de Estados REQ-05 e REQ-08	X			
Integração, testes validação dos níveis			X	X
Lógica do feedback e aleatorização			X	X

Requisito	Semana 1	Semana 2	Semana 3	Semana 4
REQ-07				
Coleta e exibição de métricas nos displays REQ-06			X	X
Impressão dos manuais do projeto				X
Disponibilidade de materiais e recursos	X	X	X	

3 Semana 1 de Desenvolvimento do Projeto

O planejamento dessa semana contempla a configuração inicial da placa FPGA com para prepará-la para a integração com os outros circuitos externos necessários para o desenvolvimento desse projeto. Para isso, será utilizada a linguagem de descrição de hardware Verilog e a ferramenta Intel Quartus Prime. O projeto foca na estruturação do sistema em duas grandes unidades: o Fluxo de Dados, responsável pelo processamento e armazenamento das informações, e a Unidade de Controle, responsável pela coordenação das operações e transições de estados. O circuito será validado no ModelSim e a implementação final será feita na FPGA DE0-CV, permitindo a validação física do circuito através de testes em bancada.

3.1 Descrição Funcional do Projeto para a Semana

O projeto para a semana 1 é uma alteração do circuito já trabalhado na Experiência 5, caminhando para um circuito capaz de ler aleatoriamente uma ROM contendo uma palavra. A palavra será composta por até endereços de 4 bits cada, a serem enviados para decodificação no computador por meio de um protocolo de comunicação ainda não definido pelo grupo. O funcionamento do sistema obedece às seguintes especificações técnicas:

- **Interface e Interação:** O jogo é iniciado através do sinal jogar. O usuário deve inserir a sequência correta através dos botões (botões), sendo que cada acionamento ocorre obrigatoriamente após pressionar o botão de confirmação de jogada, registrando-a e comparado imediatamente com o conteúdo da memória ROM interna. Qualquer partida pode ser interrompida a partir da ativação do sinal de reset.

- **Modos de Operação:** O sistema permite a seleção de dois modos distintos: Em um dos modos, a palavra inteira é enviada para o computador externo e é lida por ele de uma vez. Já no outro, as letras que compõem a palavra são enviadas sequencialmente, na ordem correta, toda vez que o usuário acertar a letra anterior.

- **Progressão:** O jogo não fará, propositalmente a escolha de palavras mais ou menos difíceis, a ideia é que todas sigam uma complexidade uniforme e o tamanho do desafio seja selecionado pelos modos de operação.

- **Condições de Derrota:** O ciclo é interrompido, ativando o sinal pedido, caso o usuário insira um dado incorreto e aperte o botão de confirmar.

- **Feedback e Depuração:** Além dos sinais de estado final (ganhou, perdeu, pronto), o projeto inclui sinais de depuração (db_contagem, db_memoria, db_estado) mapeados para displays de sete segmentos, permitindo a visualização em tempo real do estado da Unidade de Controle e do progresso do usuário na sequência.

3.2 Detalhamento do Projeto Lógico do Hardware Programável

3.2.1 Projeto do Fluxo de Dados

Em relação ao fluxo de dados, foram adicionados 4 blocos de memória ROM síncronos, um multiplexador 4x1, além do sinal de entrada confirmar, que é conectado diretamente no edge_detector, para garantir que uma nova jogada seja identificada somente por esse sinal, e não mais por qualquer alteração nas chaves de jogada do usuário. Também foi adicionado o sinal de entrada: confirmar.

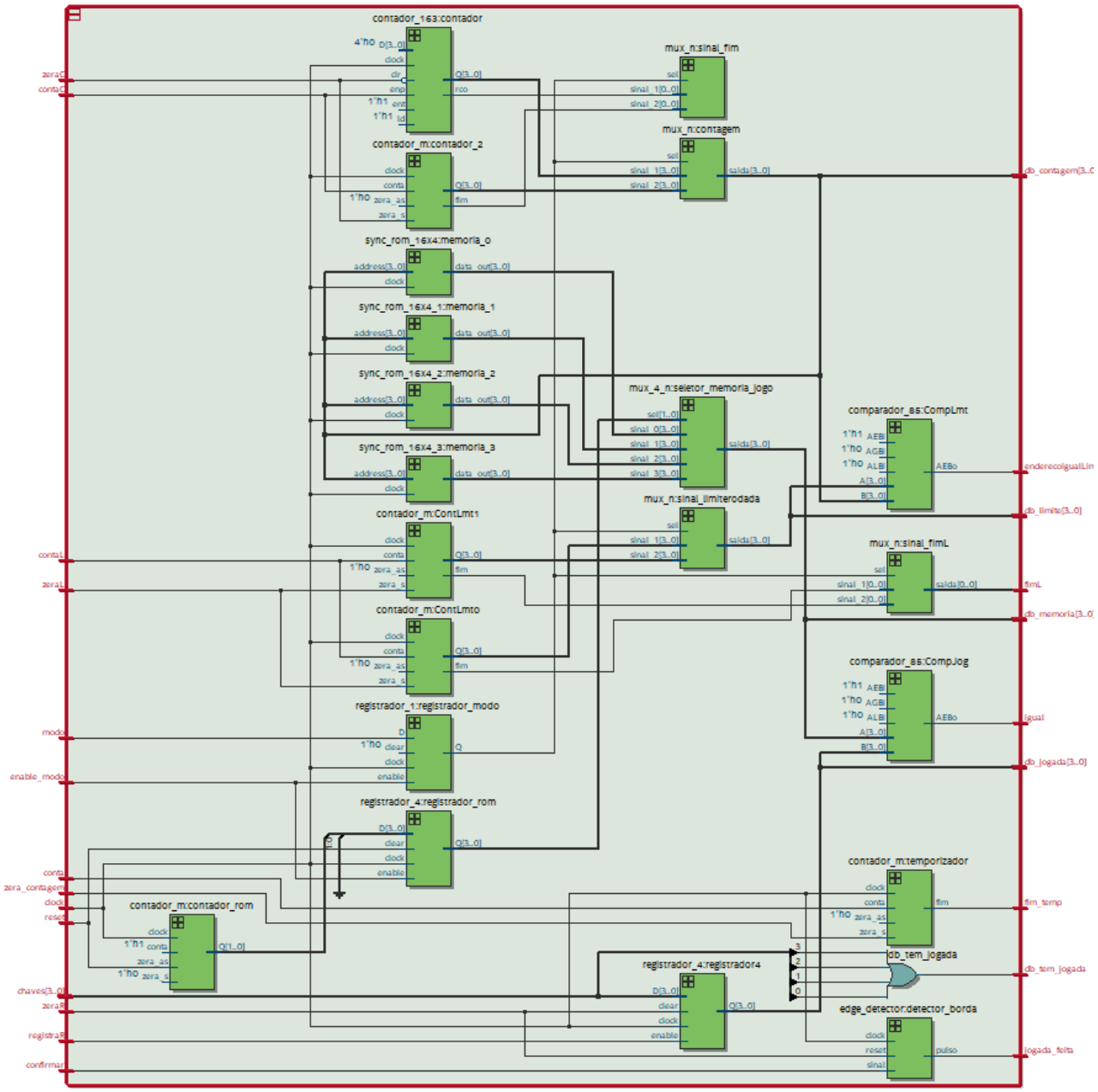

Figura 9: Fluxo de dados (RTL Viewer)

3.2.2 Projeto da Unidade de Controle

Não foram feitas alterações na unidade de controle em relação ao projeto base da experiência 5. Dessa forma, tanto os estados quanto a lógica de transição entre eles continuam as mesmas.

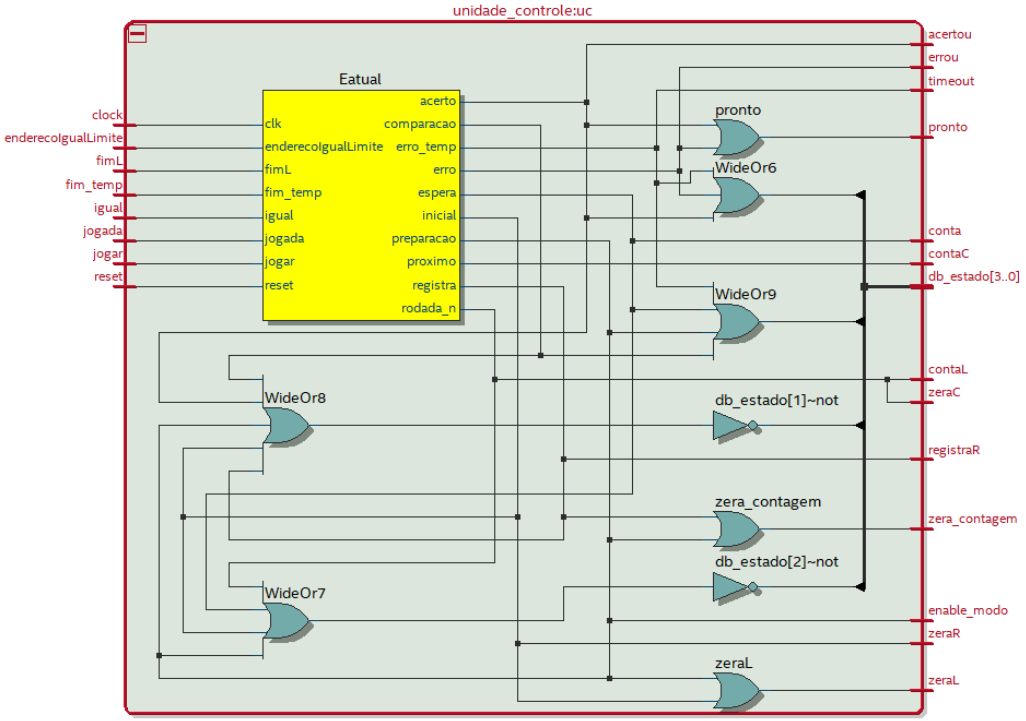

Figura 10: Unidade de Controle (RTL Viewer)

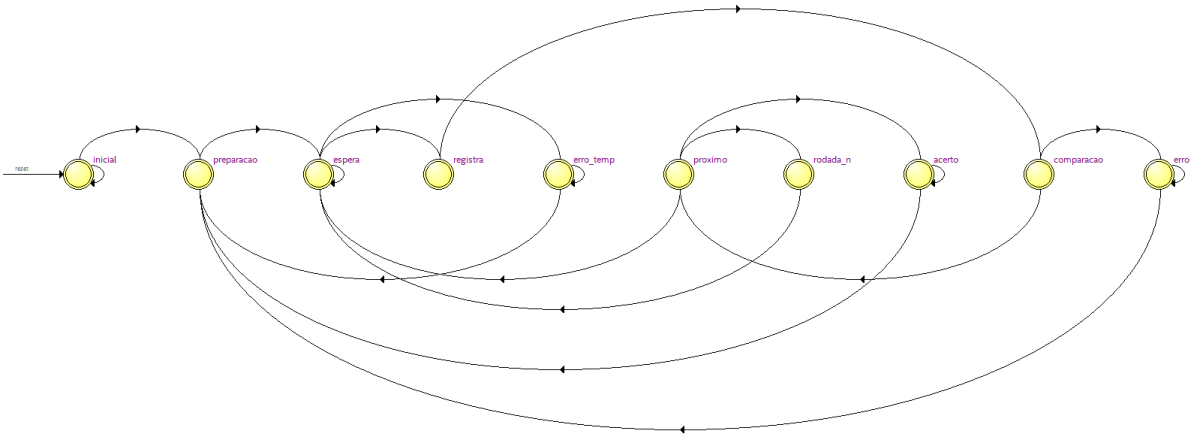

Figura 11: Máquina de Estados Finita (MEF) gerada pelo State Machine Viewer do Quartus.

3.2.3 Projeto do Sistema Digital Programável

Foi adicionado o sinal externo ¨confirmar¨ que corresponderá ao botão acionado pelo usuário para enviar sua jogada ao registrador.

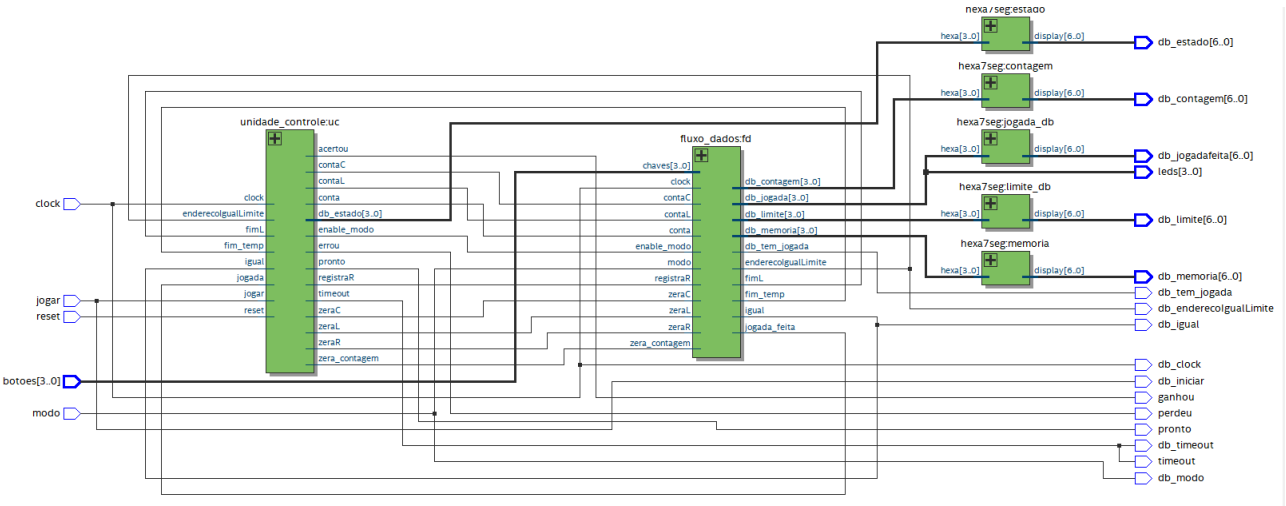

Figura 12: Diagrama de blocos do top level gerado pelo RTLViewer

3.3 Plano de Testes do Hardware Programável e Simulações

Como grande parte da experiência 5 foi reaproveitada para esse projeto e o objeto de testes desta semana será a aleatorização do circuito, não faz sentido a criação de testbenches para verificar o funcionamento do circuito, uma vez que isso já foi validado anteriormente. Com isso, serão feitos os testes para mostrar esse comportamento aleatório de seleção da ROM do circuito, bem como o funcionamento do ¨confirmador¨ de jogadas.

3.3.1 Cenário de Teste 1 – Aleatorização de escolha da ROM

Caso de teste para demonstrar a escolha aleatória da memória ROM baseada no momento em que o usuário aperta o botão de iniciar.

Tabela 14 – Descrição e Resultados Simulados do Cenário de Teste 1

Teste	Sinais de entrada			Modo	Resultado Esperado	Resultado Observado
	reset	iniciar	chaves			
C.I	0	0	0000		-	Ok
1	1	-	-	1	-	Ok

2	0	-	-	1	-	Ok
3	-	1	-	1	saída db_iniciar ativada	Ok
4	-	0	-	-	seletor_rom_out = 11	Ok
5	1	0	-	-	seletor_rom_out = 00	Ok
6	-	1	-	1	saída db_iniciar ativada	Ok
7					seletor_rom_out = 10	Ok

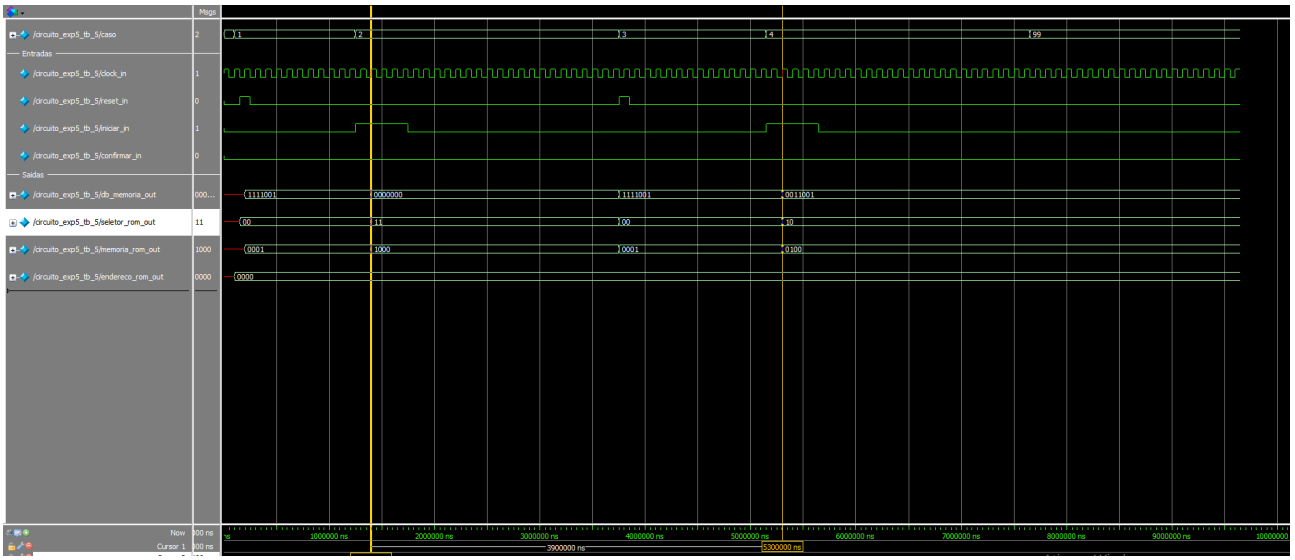

Figura 13: Cursor posicionado indicado mostrando seleção de memória da primeira partida (11) e da segunda (10).

Como é possível observar, essa simulação valida a aleatoriedade da escolha da memória ao início de cada partida. Apenas reforçando, os valores de preenchimento da tabela de testes se referem exclusivamente a essa simulação executada, realizando ela

com os mesmos sinais de entrada e na mesma sequência, nada garante que a saída será idêntica a essa.

3.3.2 Cenário de Teste 2 - Validação do sinal confirma ao executar jogadas

A entrada será alterada por meio das chaves, e após alguns ciclos de clock será ativado o sinal confirmar. Com isso, espera-se que ao final da simulação o sistema tenha registrado a jogada somente após o sinal de confirmação.

Tabela 15 – Descrição e Resultados Simulados do Cenário de Teste 2

Teste	Sinais de entrada			confirmar	Resultado Esperado	Resultado Observado
	reset	iniciar	chaves			
C.I	0	0	0000	0	-	Ok
1	1	-	-	-	-	Ok
2	0	-	-	-	-	Ok
3	-	1	-	-	saída db_iniciar ativada	Ok
4	-	0	0010	-	jogada_reg_out = 0000	Ok
5	1	0	0000	-	jogada_reg_out = 0000	Ok
6	-	0	0010	-	jogada_reg_out = 0000	Ok
7	-	-	-	1	jogada_reg_out = 0010	Ok

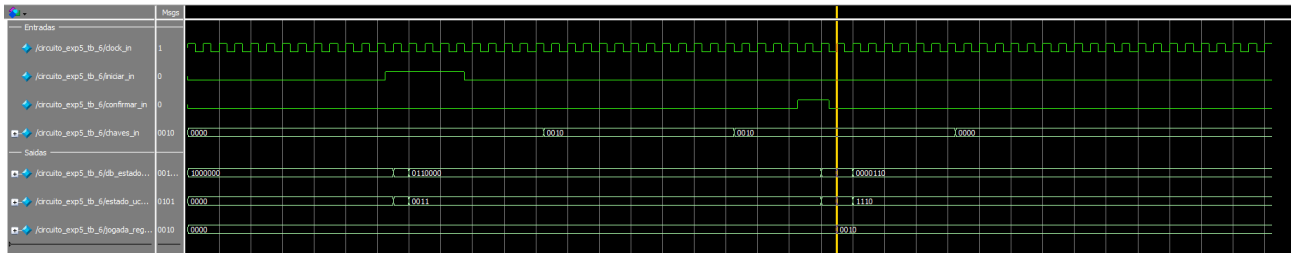

Figura 14: Forma de onda mostrando o registro da jogada somente após o sinal de confirmar

Como mostrado, o sinal de confirmar de fato cumpre com o comportamento especificado no projeto.

3.4 Detalhamento do Projeto de Software

3.4.1 Descrição da Arquitetura de Software

3.4.2 Detalhamento do Projeto de Software

3.5 Plano de Testes do Software

3.5.1 Cenário de Teste X – <Objetivo Sucinto do Cenário>

3.6 Integração entre Hardware Programável e Software[1]

3.7 Prototipação de Projeto

Continua seguindo as mesmas instruções de montagem que as já presentes no planejamento/relato da experiência 5. A única alteração está no pin planner, onde o botão correspondente ao sinal de confirmar foi alocado para o pino PIN_M6.

3.8 Implantação do Projeto

3.8.1 Pinagem da Placa FPGA

[1] As seções referentes a softwares não se aplicam à semana atual, pois o grupo não desenvolveu essa parte do projeto

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate
botoes[3]	Input	PIN_K17	5B	B5B_N0	PIN_K17	2.5 V		12mA (default)	
botoes[2]	Input	PIN_L17	5B	B5B_N0	PIN_L17	2.5 V		12mA (default)	
botoes[1]	Input	PIN_M18	5B	B5B_N0	PIN_M18	2.5 V		12mA (default)	
botoes[0]	Input	PIN_P17	5A	B5A_N0	PIN_P17	2.5 V		12mA (default)	
clock	Input	PIN_R22	5A	B5A_N0	PIN_R22	2.5 V		12mA (default)	
confirmar	Input	PIN_M6	3A	B3A_N0	PIN_V20	2.5 V (default)		12mA (default)	
db_clock	Output	PIN_N1	2A	B2A_N0	PIN_N1	2.5 V		12mA (default)	1 (default)
db_contagem[6]	Output	PIN_AA22	4A	B4A_N0	PIN_AA22	2.5 V		12mA (default)	1 (default)
db_contagem[5]	Output	PIN_Y21	4A	B4A_N0	PIN_Y21	2.5 V		12mA (default)	1 (default)
db_contagem[4]	Output	PIN_Y22	4A	B4A_N0	PIN_Y22	2.5 V		12mA (default)	1 (default)
db_contagem[3]	Output	PIN_W21	4A	B4A_N0	PIN_W21	2.5 V		12mA (default)	1 (default)
db_contagem[2]	Output	PIN_W22	4A	B4A_N0	PIN_W22	2.5 V		12mA (default)	1 (default)
db_contagem[1]	Output	PIN_V21	4A	B4A_N0	PIN_V21	2.5 V		12mA (default)	1 (default)
db_contagem[0]	Output	PIN_U21	4A	B4A_N0	PIN_U21	2.5 V		12mA (default)	1 (default)
db_endere...ualLimite	Output	PIN_L1	2A	B2A_N0	PIN_L1	2.5 V		12mA (default)	1 (default)
db_estado[6]	Output	PIN_W19	4A	B4A_N0	PIN_W19	2.5 V		12mA (default)	1 (default)
db_estado[5]	Output	PIN_C2	2A	B2A_N0	PIN_C2	2.5 V		12mA (default)	1 (default)
db_estado[4]	Output	PIN_C1	2A	B2A_N0	PIN_C1	2.5 V		12mA (default)	1 (default)
db_estado[3]	Output	PIN_P14	4A	B4A_N0	PIN_P14	2.5 V		12mA (default)	1 (default)
db_estado[2]	Output	PIN_T14	4A	B4A_N0	PIN_T14	2.5 V		12mA (default)	1 (default)
db_estado[1]	Output	PIN_M8	3B	B3B_N0	PIN_M8	2.5 V		12mA (default)	1 (default)
db_estado[0]	Output	PIN_N9	3B	B3B_N0	PIN_N9	2.5 V		12mA (default)	1 (default)
db_igual	Output	PIN_L2	2A	B2A_N0	PIN_L2	2.5 V		12mA (default)	1 (default)
db_iniciar	Output	PIN_U1	2A	B2A_N0	PIN_U1	2.5 V		12mA (default)	1 (default)
db_limite[6]	Output	PIN_V19	4A	B4A_N0	PIN_V19	2.5 V		12mA (default)	1 (default)
db_limite[5]	Output	PIN_V18	4A	B4A_N0	PIN_V18	2.5 V		12mA (default)	1 (default)
db_limite[4]	Output	PIN_U17	4A	B4A_N0	PIN_U17	2.5 V		12mA (default)	1 (default)
db_limite[3]	Output	PIN_V16	4A	B4A_N0	PIN_V16	2.5 V		12mA (default)	1 (default)
db_limite[2]	Output	PIN_Y17	4A	B4A_N0	PIN_Y17	2.5 V		12mA (default)	1 (default)
db_limite[1]	Output	PIN_W16	4A	B4A_N0	PIN_W16	2.5 V		12mA (default)	1 (default)
db_limite[0]	Output	PIN_Y16	4A	B4A_N0	PIN_Y16	2.5 V		12mA (default)	1 (default)
db_memoria[6]	Output	PIN_U22	4A	B4A_N0	PIN_U22	2.5 V		12mA (default)	1 (default)
db_memoria[5]	Output	PIN_AA17	4A	B4A_N0	PIN_AA17	2.5 V		12mA (default)	1 (default)
db_memoria[4]	Output	PIN_AB18	4A	B4A_N0	PIN_AB18	2.5 V		12mA (default)	1 (default)
db_memoria[3]	Output	PIN_AA18	4A	B4A_N0	PIN_AA18	2.5 V		12mA (default)	1 (default)
db_memoria[2]	Output	PIN_AA19	4A	B4A_N0	PIN_AA19	2.5 V		12mA (default)	1 (default)
db_memoria[1]	Output	PIN_AB20	4A	B4A_N0	PIN_AB20	2.5 V		12mA (default)	1 (default)
db_memoria[0]	Output	PIN_AA20	4A	B4A_N0	PIN_AA20	2.5 V		12mA (default)	1 (default)
db_modo	Output	PIN_N2	2A	B2A_N0	PIN_N2	2.5 V		12mA (default)	1 (default)
db_tem_jogada	Output	PIN_U2	2A	B2A_N0	PIN_U2	2.5 V		12mA (default)	1 (default)
db_timeout	Output				PIN_U6	2.5 V (default)		12mA (default)	1 (default)
ganhou	Output	PIN_AA1	2A	B2A_N0	PIN_AA1	2.5 V		12mA (default)	1 (default)
jogar	Input	PIN_N19	5B	B5B_N0	PIN_N19	2.5 V		12mA (default)	
leds[3]	Output	PIN_T18	5A	B5A_N0	PIN_T18	2.5 V		12mA (default)	1 (default)
leds[2]	Output	PIN_T20	5A	B5A_N0	PIN_T20	2.5 V		12mA (default)	1 (default)
leds[1]	Output	PIN_R17	5A	B5A_N0	PIN_R17	2.5 V		12mA (default)	1 (default)
leds[0]	Output	PIN_P18	5A	B5A_N0	PIN_P18	2.5 V		12mA (default)	1 (default)
modo	Input	PIN_P19	5A	B5A_N0	PIN_P19	2.5 V		12mA (default)	
perdeu	Output	PIN_AA2	2A	B2A_N0	PIN_AA2	2.5 V		12mA (default)	1 (default)
pronto	Output	PIN_Y3	2A	B2A_N0	PIN_Y3	2.5 V		12mA (default)	1 (default)
reset	Input	PIN_T22	5A	B5A_N0	PIN_T22	2.5 V		12mA (default)	

Figura 15: Pin Planner parte 1

jogar	Input	PIN_N19	5B	B5B_N0	PIN_N19	2.5 V		12mA (default)	
leds[3]	Output	PIN_T18	5A	B5A_N0	PIN_T18	2.5 V		12mA (default)	1 (default)
leds[2]	Output	PIN_T20	5A	B5A_N0	PIN_T20	2.5 V		12mA (default)	1 (default)
leds[1]	Output	PIN_R17	5A	B5A_N0	PIN_R17	2.5 V		12mA (default)	1 (default)
leds[0]	Output	PIN_P18	5A	B5A_N0	PIN_P18	2.5 V		12mA (default)	1 (default)
modo	Input	PIN_P19	5A	B5A_N0	PIN_P19	2.5 V		12mA (default)	
perdeu	Output	PIN_AA2	2A	B2A_N0	PIN_AA2	2.5 V		12mA (default)	1 (default)
pronto	Output	PIN_Y3	2A	B2A_N0	PIN_Y3	2.5 V		12mA (default)	1 (default)
reset	Input	PIN_T22	5A	B5A_N0	PIN_T22	2.5 V		12mA (default)	
timeout	Output	PIN_W2	2A	B2A_N0	PIN_W2	2.5 V		12mA (default)	1 (default)
db_jogada[0]	Unknown	PIN_Y19	4A	B4A_N0		2.5 V (default)		12mA (default)	
db_jogada[1]	Unknown	PIN_AB17	4A	B4A_N0		2.5 V (default)		12mA (default)	
db_jogada[2]	Unknown	PIN_AA10	3B	B3B_N0		2.5 V (default)		12mA (default)	
db_jogada[3]	Unknown	PIN_Y14	4A	B4A_N0		2.5 V (default)		12mA (default)	
db_jogada[4]	Unknown	PIN_V14	4A	B4A_N0		2.5 V (default)		12mA (default)	
db_jogada[5]	Unknown	PIN_AB22	4A	B4A_N0		2.5 V (default)		12mA (default)	
db_jogada[6]	Unknown	PIN_AB21	4A	B4A_N0		2.5 V (default)		12mA (default)	

Figura 16: Pin Planner parte 2

3.8.2 Estratégia de Montagem

a) Programação da placa FPGA

Como usual, a placa FPGA será conectada ao computador do laboratório por meio de um cabo USB (servirá tanto como alimentação, quanto para transferência de dados), e a programação da placa será feita utilizando a ferramenta ¨Programmer¨ do Inter Quartus.

b) Ligações de terminais e fios

A conexão principal será realizada entre a FPGA e o computador da bancada utilizando um cabo USB. Ademais, para uso do dispositivo AnalogDiscovery, será necessário interligar os equipamentos com o uso de jumpers da seguinte forma:

GND: Pino 12

Clock: Pino 14

Reset: Pino 16

Botões (0 a 3): Pinos 22,24,26 e 28, respectivamente

Leds (0 a 3): Pinos 32,34, 36 e 38, respectivamente

Iniciar: Pino 18

Modo: Pino 20

Obs. 1: Sempre ligar primeiramente o terra (GND) e depois dar seguimento às outras conexões.

Obs. 2: A pinagem descrita acima se refere exclusivamente ao barramento GPIO–0.

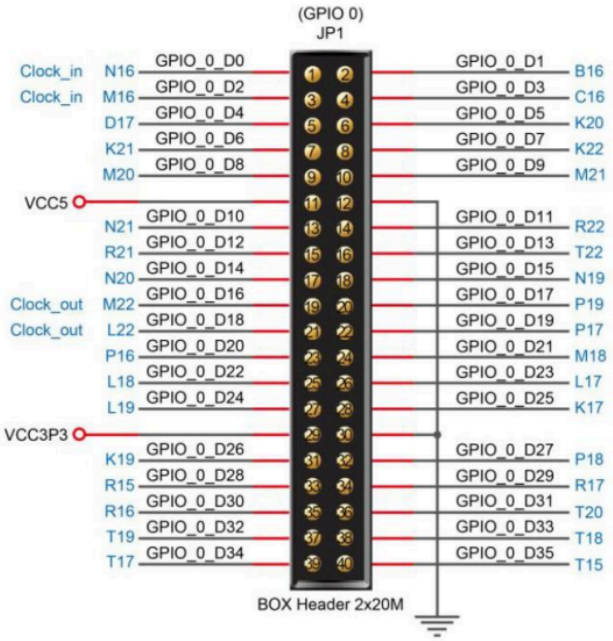

Figura 17: Ilustração dos conectores do GPIO-0 da placa DE0-CV com numeração

c) Sinais/elementos físicos monitorados

Os elementos físicos da placa que serão utilizados para interagir e monitorar o sistema são:

a) Entradas: Todos os pinos GPIO conectados no Analog Discovery através do software waveform.

b) Saídas binárias: LEDS 0 a 9

c) Saídas de dados: Displays de sete segmentos 0,1,2 e 5

d) Sinais de depuração considerados e como são monitorados

Os sinais considerados serão explicitados no item 3.8.3a do relatório e estes serão monitorados a partir dos leds e displays mencionados anteriormente (item 3.8.2.c).

3.8.3 Estratégia de Depuração

a) **Sinais de depuração:**

Foram incluídos os sinais de depuração da experiência anterior:

a) db_estado (Saída para HEX5): Exibe o estado atual da FSM. Fundamental para verificar se o sistema está transicionando de estados de maneira coerente.

b) db_contagem (Saída para HEX0): Monitora o endereço de memória atual. Permite validar se o contador está incrementando sequencialmente e parando no momento certo.

c) db_memoria (Saída para HEX1): Exibe o dado armazenado na ROM, o que é fundamental para conferir a lógica de comparação do circuito.

d) db_jogada (Saída para HEX2): Mostra o valor que foi efetivamente registrado no registrador_4. Permite verificar se a leitura do dado inserido pelas chaves foi correta.

e) db_tem_jogada, db_clock, db_igual e db_iniciar (Saídas para LEDs): Indicadores de atividade (heartbeat). O db_tem_jogada confirma se o sinal elétrico das chaves está chegando ao edge detector, enquanto o db_clock confirma a integridade do sinal de clock externo. Por último, db_igual indica se o valor das chaves é igual ao da memória.

f) LEDs: Embora a verificação do valor da memória já seja feita pelo display de 7 segmentos em hexadecimal, os leds também representarão esse valor em binário do valor da memória.

Além deles, também foram incluídos outros 4 sinais de depuração para essa experiência:

g) db_modo e db_enderecoIgualLimite (saída para leds): db_modo indica o modo escolhido. Importante para verificar se há efeito no circuito ao alterar o modo durante a partida (o que não deveria ocorrer). Enquanto isso, db_enderecoIgualLimite mostra o indica quando o sinal de condição enderecoIgualLimite está ativado.

i) db_limite (Saída para HEX3) : indica o valor do contador de limite da rodada atual.

g) Monitoramento dos sinais

Os sinais serão monitorados visualmente a partir dos leds e displays de 7 segmentos da placa FPGA.

h) Possíveis alterações no projeto lógico

Caso necessário, as alterações serão feitas nos códigos verilog dos componentes utilizados e, com isso, o circuito será novamente compilado e sintetizado para, enfim, reprogramar a placa.

i) Possíveis alterações na montagem

Se necessário, pode-se alterar a pinagem dos componentes da placa, sempre em conformidade com a tabela de pinos para a placa DE0-CV disponibilizada na página da disciplina no e-disciplinas.

j) Testes de regressão

Para qualquer alteração feita, seja na lógica do circuito ou na montagem, todos os testes serão reexecutados, podendo ainda adicionar mais casos de testes, para garantir o correto funcionamento do circuito em todos os aspectos.

3.8.4 Execução Prática do Cenário de Teste 1 – Aleatorização de escolha da ROM

Caso de teste para demonstrar a escolha aleatória da memória ROM baseada no momento em que o usuário aperta o botão de iniciar.

Tabela 16 – Descrição e Resultados Práticos do Cenário de Teste 1

Teste	Sinais de entrada			Modo	Resultado Esperado	Resultado Observado
	reset	iniciar	chaves			
C.I	0	0	0000		-	Ok
1	1	-	-	1	-	Ok
2	0	-	-	1	-	Ok
3	-	1	-	1	saída db_iniciar ativada	Ok
4	-	0	-	-	seletor_rom_out = 11	Ok
5	1	0	-	-	seletor_rom_out = 00	Ok
6	-	1	-	1	saída db_iniciar ativada	Ok

7					seletor_rom_out = 10	Ok

Pode-se observar que a aleatorização na escolha da ROM funcionou conforme o esperado, como se pode observar no cenário gravado a seguir: Cenário de teste 1

3.8.5 Execução Prática do Cenário de Teste 2 - Validação do sinal confirma ao executar jogadas

A entrada será alterada por meio das chaves, e após alguns ciclos de clock será ativado o sinal confirmar. Com isso, espera-se que ao final da simulação o sistema tenha registrado a jogada somente após o sinal de confirmação.

Tabela 17 – Descrição e Resultados Práticos do Cenário de Teste 2

Teste	Sinais de entrada			confirmar	Resultado Esperado	Resultado Observado
	reset	iniciar	chaves			
C.I	0	0	0000	0	-	Ok
1	1	-	-	-	-	Ok
2	0	-	-	-	-	Ok
3	-	1	-	-	saída db_iniciar ativada	Ok
4	-	0	0010	-	jogada_reg_out = 0000	Ok
5	1	0	0000	-	jogada_reg_out = 0000	Ok
6	-	0	0010	-	jogada_reg_out = 0000	Ok

7	-	-	-	1	jogada_reg_out = 0010	Ok

Assim como na execução prática do teste 1, esse cenário de teste foi corretamente comprovado e verificado pela gravação: Cenário de teste 2

3.9 Conclusão da Semana

Com um intervalo curto de tempo entre a AEX4 e o início da primeira semana do projeto, algumas lacunas de implementação principalmente em relação à FPGA - que era o foco da semana - ficaram pendentes. Contudo, foi possível implementar com sucesso o que o grupo havia proposto no planejamento e no *pitch* da semana.

Nesse contexto, durante o laboratório, os testes de aleatoriedade na escolha da memória e o botão de confirmar obtiveram pleno sucesso simulado e prático. No tempo extraclasse, uma primeira leva de compras com os componentes principais foi iniciada e algumas tarefas pendentes foram subdivididas entre os membros para serem implementadas anteriormente à semana 2, são elas: o estudo da comunicação serial via UART e a modificação na arquitetura e funcionamento da ROM para leitura correta das palavras.

4 Semana 2 de Desenvolvimento do Projeto

Para a semana 2, o planejamento consiste em refatorar o código por completo, eliminando os resquícios da arquitetura referente ao projeto da experiência 5, garantindo que este esteja 100% aderente a lógica proposta para dar continuidade ao desenvolvimento do jogo SomLetrando. As ferramentas continuam as mesmas: linguagem de descrição de hardware Verilog e o software Inter Quartus Prime. A estrutura também continua a mesma: UC e FD ligados pela entidade topo. Além disso, houve uma tentativa de realizar o processo de comunicação serial utilizando o Analog Discovery.

4.1 Descrição Funcional do Projeto para a Semana

O comportamento do circuito se mantém essencialmente o mesmo da semana 1, entretanto, foi necessário a criação de uma nova arquitetura para as memórias ROM seguindo os seguintes requisitos:

- Todas as palavras devem ficar em uma única ROM em que cada letra ocupará um endereço de memória;
- Registrar o endereço inicial de cada palavra;
- A escolha da palavra deverá ser aleatória;
- O fim de cada palavra será representado por um caractere de parada, o “0000”;
- Ao errar uma letra da palavra o sistema deve continuar na mesma letra até que o usuário acerte a jogada ou o limite de erros seja atingido.
- Em caso de o usuário errar 3 vezes a partida será encerrada como derrota

Foram realizados trabalhos em paralelos pelo grupo para o desenvolvimento do projeto nessa semana: um integrante ficou responsável pela nova arquitetura das memórias ROM (como descrito acima) enquanto outro integrante iniciou o estudo da comunicação serial através do Analog Discovery. Nesse processo, para fins **somente de cenários de teste** criado pelo primeiro integrante**,** a ROM utiliza caracteres de 4 bits com significado arbitrário (Não foi adotada nenhuma correspondência entre este valores binários de 4 bits e as letras do alfabeto). Porém, o segundo integrante que integrou a comunicação serial, utilizou a notação ASCII para representar os caracteres que compõem as letras, deixando a ROM no tamanho adequado de 7 bits.

4.2 Detalhamento do Projeto Lógico do Hardware Programável

4.2.1 Projeto do Fluxo de Dados

Para que os requisitos definidos na seção 4.1 fossem atendidos os seguintes componentes foram adicionados ao fluxo de dados:

- **rom_ponteiros**: Armazena, em ordem, os endereços iniciais de cada palavra na memória;
- **rom_palavras**: Todas as palavras estarão pré-gravadas uma após a outra sempre separadas pelo caractere de parada 00000000;
- **endereco_palavra**: Ao início de cada partida será feito um load síncrono para começar a contagem do primeiro endereço da palavra escolhida de forma aleatória, os incrementos serão realizados após cada jogada certa até encontrar o caractere de parada;

- **CompLmt**: Comparador para verificação do caractere de parada e gerar sinal de condição para ser enviado a Unidade de controle.
- **contador_erros**: Responsável pela contagem das jogadas erradas do usuário e enviar o sinal de condição limite_erros para a Unidade de Controle.

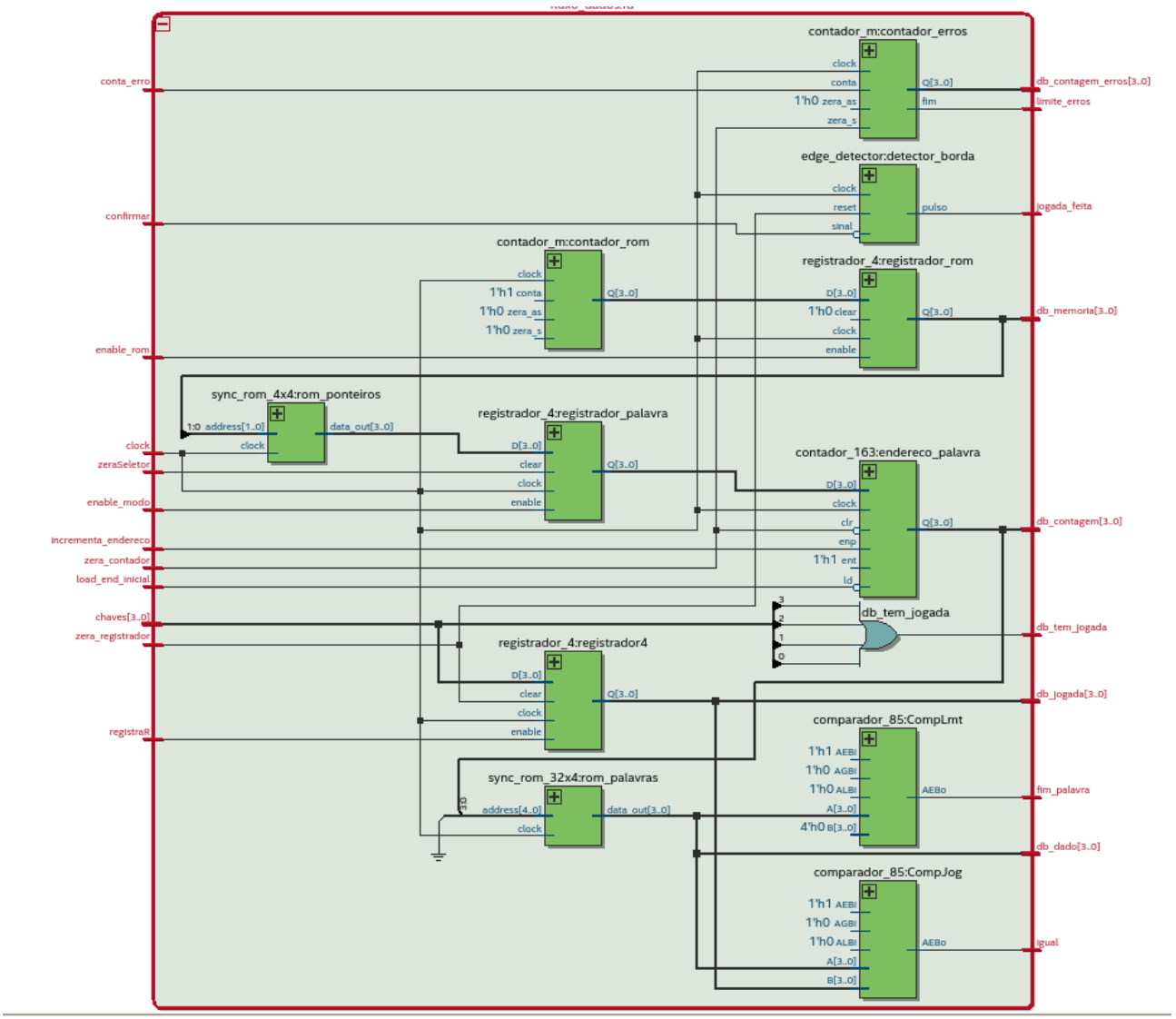

Figura 18: Diagrama de blocos do Fluxo de Dados gerado pelo RTLViewer

4.2.2 Projeto da Unidade de Controle

Para a unidade de controle, o principal desafio encontrado foi no que diz respeito à temporização do circuito. Como todos os componentes de armazenamento utilizados são síncronos, gerar um sinal de controle e esperar um sinal de condição que depende diretamente deste sinal da UC pode gerar comportamento inesperado. Dessa forma, foi necessário a criação de estados intermediários extras para garantir que ações resultantes dos sinais de controle tivessem o tempo necessário para gerar suas respectivas saídas. A tabela abaixo descreve o papel de cada estado no circuito, bem como suas condições de transição de estados. Para complementar a visualização do projeto lógico resultante,

também é possível observar o desenho do diagrama de alto nível do circuito na figura abaixo.

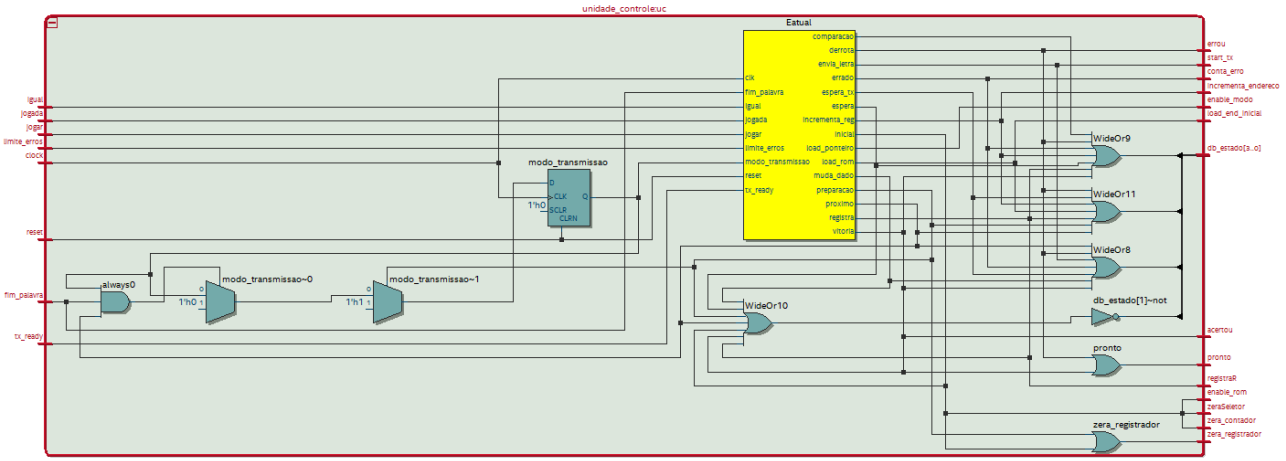

Figura 19: Diagrama de blocos UC

TABELA 18: DESCRIÇÃO E CONDIÇÕES DE TRANSIÇÃO DOS ESTADOS DA FSM DO SISTEMA

Nome do Estado	Descrição do Estado	Próximo Estado	Condições e Justificativas para a Transição entre Estados
Inicial	Estado inicial do circuito logo ao ser ligado. Zera o contador e o registrador.	Preparacao ou inicial	Se jogar = 1: Eprox <= Preparacao Se jogar = 0: Eprox <= Inicial
Preparacao	Estado para garantir que o contador e o registrador estão zerados	load_ponteiro	Sem condições de transição
load_ponteiro	Estado para garantir que o registrador do ponteiro de memória foi corretamente armazenado	load_rom	Sem condições de transição
load_rom	Estado para garantir que o enderço inicial da palavra foi corretamente carregado no contador.	Envia_letra ou Espera	Se modo_transmissao = 1: Eprox <= envia_letra Se modo_transmissao = 0: Eprox <= Espera
envia_letra	Inicia comunicação serial	Espera_tx	Sem condições de transição
espera_tx	Aguarda fim da	incrementa_	Se tx_ready = 1:

Nome do Estado	Descrição do Estado	Próximo Estado	Condições e Justificativas para a Transição entre Estados
	comunicação serial	reg ou espera_tx	Eprox <= incrementa_reg Se tx_ready = 0: Eprox <= espera_tx
Espera	Estado de espera para jogada do usuário	Espera ou Registra	Se jogada = 1: Eprox <= Registra Se jogada = 0: Eprox <= Espera
Registra	Registra o valor entrado nas chaves pelo usuário	Comparacao	Sem condições de transição
Comparacao	Comparar o valor das chaves no registrador com o respectivo endereço de memória selecionado pelo contador	Errardo ou Incrementa_ reg	Se igual == 0: Eprox <= Errado Se igual = 1: Incrementa_reg
Incrementa_r eg	Estado exclusivamente para incremento do contador de endereços da palavra escolhida	Proximo	Sem condições de transição
errado	Estado para incrementar o contador de erros do	derrota ou Espera	Se limite_erros = 0: Eprox <= Espera Se limite_erros = 1: Eprox <= Derrota
Proximo	Estado de verificação de condição de vitória do usuário e de transmissão	Espera ou Vitoria ou load_rom ou envia_letra	Se modo_transmissão = 1: se fim_palavra = 1: Eprox <= load_rom se fim_palavra = 0: envia_letra Se modo_transmissão = 0: se fim_palavra = 1: Eprox <= Vitoria se fim_palavra = 0: Eprox<= Espera
Vitoria	Estado para indicar fim da execução indicando sucesso nas comparações	Vitoria ou Inicial	Se jogar = 0: Eprox <= Vitoria Se jogar = 1: Eprox <= Inicial

Nome do Estado	Descrição do Estado	Próximo Estado	Condições e Justificativas para a Transição entre Estados
Derrota	Estado para indicar fim da execução indicando falha em algumas das comparações	Derrota ou Inicial	Se jogar = 0: Eprox <= Derrota Se jogar = 1: Eprox <= Inicial

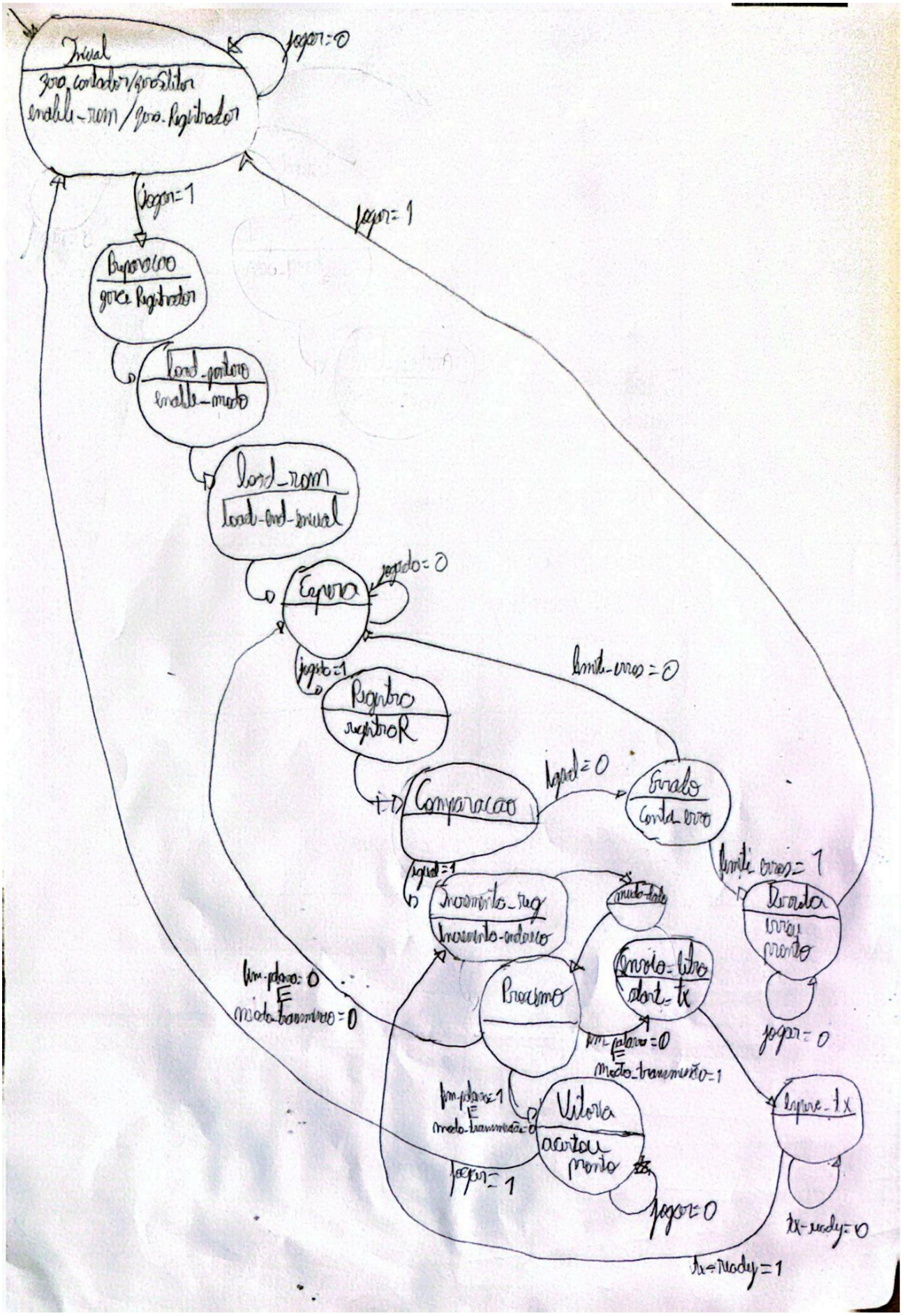
Inicial
zera_contador/zera_bits
enable_rom / zera_registrador
jogar=0
jogar=1
jogar=1
Espera
load_rom
load_end_inicial
Espera
jogado=0
jogado=1
Registro
receberR
Comparacao
Erro
conta_erro
limite_erros=0
limite_erros=1
Derrota
Proximo_rug
Proximo_endereco
Proximo
Envio_bits
Envio_bits
start_tx
Espera_tx
Vitoria
acertou
pronto
jogar=0
jogar=1
tx_ready=0
tx_ready=1

Figura 20: Diagrama de alto nível da FSM

4.2.3 Projeto do Sistema Digital Programável

Para a integração completa entre FD e UC na entidade topo foram adicionadas novas ligações referentes aos sinais de controle e condição adequados a nova arquitetura de 2 ROMs, além dos sinais específicos a lógica de contagem de erros, permitindo que o usuário erre até 3 vezes antes de encerrar a partida como derrota. Também foram adicionados sinais de depuração pertinentes à nova lógica de funcionamento do sistema e um novo módulo de transmissão serial fornecido no e-disciplinas foi integrado ao sistema.

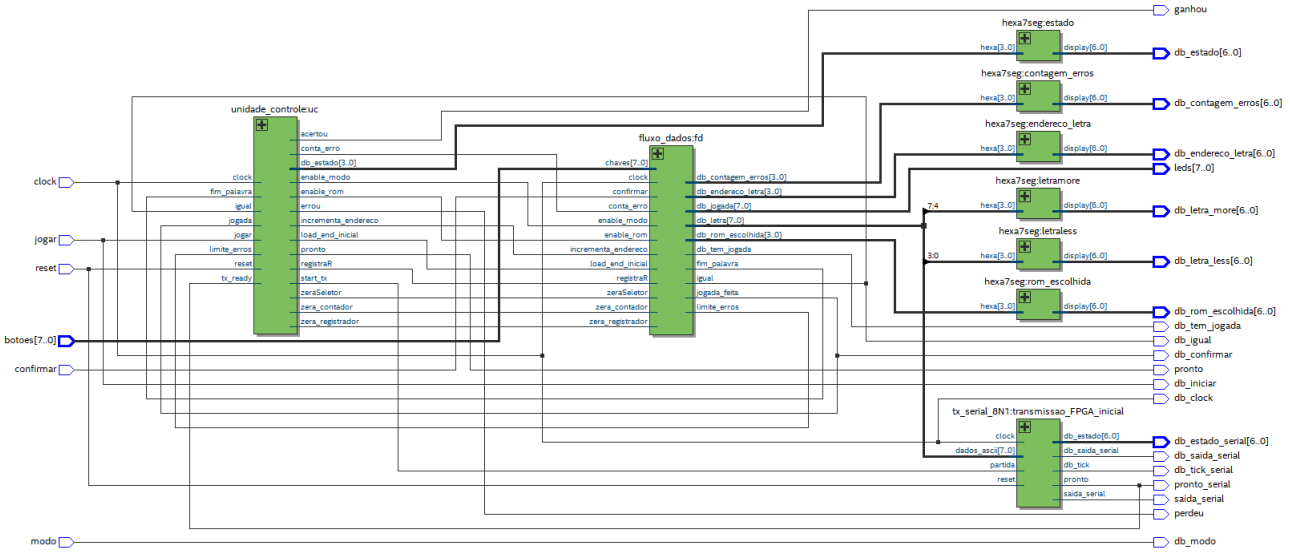

Figura 21: Diagrama de blocos da entidade topo gerada pelo RTLViewer

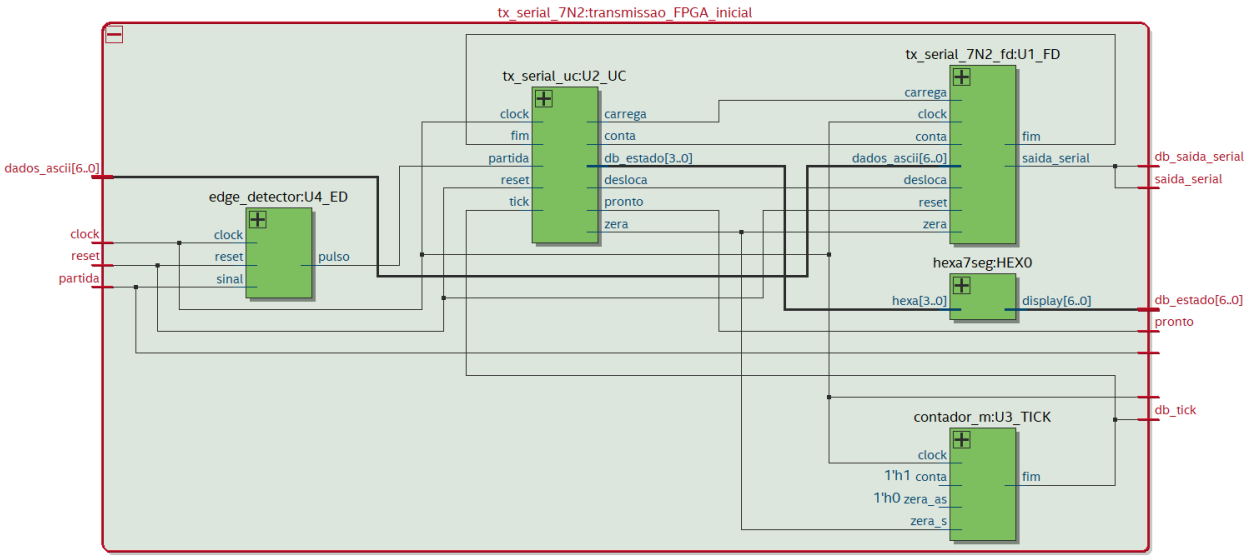

Figura 22: Diagrama de blocos do módulo de transmissão serial gerada pelo RTLViewer

4.3 Plano de Testes do Hardware Programável e Simulações

Uma grande dificuldade de utilizar simulações no ModelSim para o sistema desenvolvido pelo grupo vem da escolha aleatória de palavra, o que impossibilita saber de antemão quais serão necessários na entrada. Isso conflita diretamente com o comportamento rígido de uma tb, que exige a predefinição de todos os sinais de input para mapear o comportamento do circuito. Dessa forma, apenas testes que não são afetados pela aleatorização do circuito serão simulados, os demais serão executados na bancada do laboratório com a placa FPGA em momento oportuno (OpenLab/horário de aula).

4.3.1 Cenário de Teste 1 - Escolha aleatória de palavra

Buscar validar se as mudanças feitas em relação à semana 1 na arquitetura das ROMs mantiveram a consistência no que diz respeito à escolha aleatória de palavras.

Tabela 19: Descrição e resultados do cenário de testes 1 da semana 2

Teste	Sinais de entrada			Modo	Resultado Esperado	Resultado Observado
	reset	iniciar	chaves			
C.I	0	0	0000		-	Ok
1	1	-	-	1	-	Ok
2	0	-	-	1	-	Ok
3	-	1	-	1	saída db_iniciar ativada	Ok
4	-	0	-	-	seletor_rom_out = 00	Ok
5	1	0	-	-	seletor_rom_out = 00	Ok
6	-	1	-	1	saída db_iniciar ativada	Ok
7					seletor_rom_out = 10	Ok

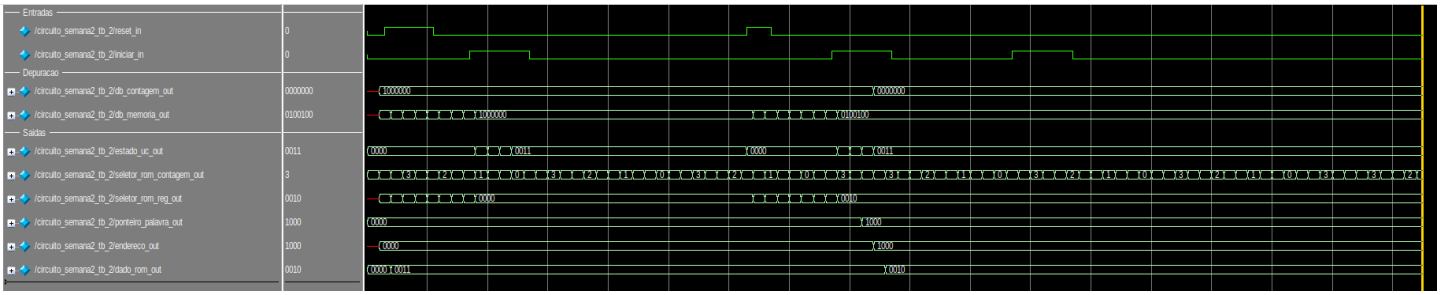

Figura 23: Forma de onda mostrando a seleção aleatória da ROM

4.4 Detalhamento do Projeto de Software

A ideia da primeira implementação de software no projeto é realizar a comunicação entre a FPGA e o computador, de modo que a lógica do SomLetrando esteja presente majoritariamente na FPGA (hardware) - que é o foco da disciplina - e o computador (software) sirva apenas como um complemento ao desenvolvimento do projeto. Nesse sentido, o SomLetrando, visando uma estimulação auditiva a crianças autistas pretende utilizar o computador meramente como reprodutor de áudio - tanto das letras quanto da palavra - logo após a palavra ter sido escolhida no banco (Figura 7) e sempre que o usuário desejar apertar a tecla de “repetir” indicada no painel de botões (Figura 5).

Para realizar essa comunicação entre as partes, o grupo propôs como solução mais imediata e menos custosa, a utilização do próprio Analog Discovery na funcionalidade de “Protocol” para transmissão serial dos bits que formam as palavras e as letras.

4.4.1 Descrição da Arquitetura de Software

Uma das maneiras mais simples de se construir uma comunicação entre microcontroladores e periféricos é o protocolo UART (Universal Asynchronous Receiver/Transmitter) justamente porque utiliza apenas um fio de transmissão (TX), um fio de recepção (RX) e um GND para converter dados paralelos em seriais (transmissão) e vice-versa apenas baseado em *start, word data* e *stop bits.*

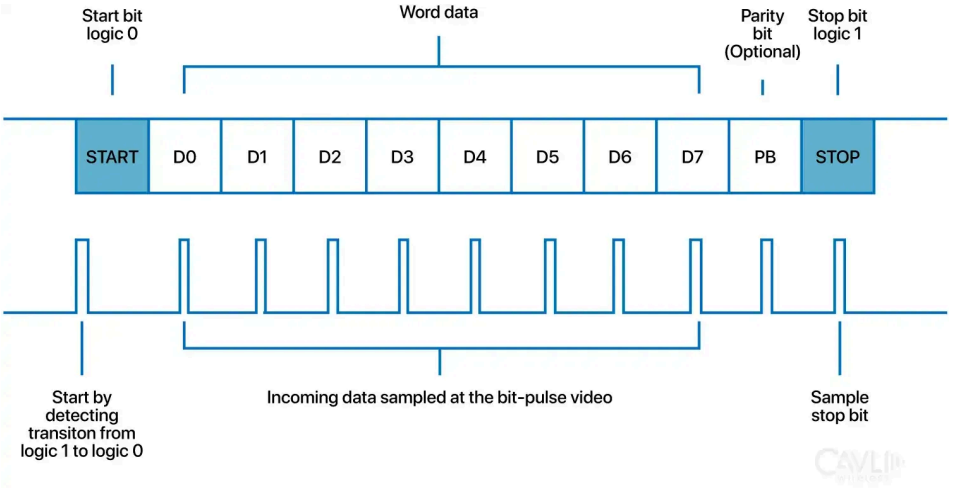

Figura 24: Funcionamento de Comunicação Serial UART (Cavli Wireless)

Para elaboração inicial da arquitetura do software foram utilizados os códigos presentes digital_UART da Waveforms SDK, uma API da *Digilent* que permite a customização própria do usuário para construção de seus códigos e visualização posterior no aplicativo Waveforms.

Initialize the System

SDK Function	Description
FDwfDeviceOpen	Open the first device it finds.

Configure the UART

SDK Function	Description
FDwfDigitalUartRateSet	Set the baud rate.
FDwfDigitalUartTxSet	Set D0 to be the transmit line.
FDwfDigitalUartRxSet	Set D1 to be the receive line.
FDwfDigitalUartBitsSet	Set the number of data bits to eight.
FDwfDigitalUartParitySet	Set no parity.
FDwfDigitalUartStopSet	Set one stop bit.
FDwfDigitalUartTx	Initialize the transmit lines to the idle state.
FDwfDigitalUartRx	Initialize the receive line.

Send and receive UART data

SDK Function	Description
FDwfDigitalUartTx	Send sample message like *Hello<cr><lf>*.
FDwfDigitalUartRx	Receive incoming data if there is any.

Shutdown

SDK Function	Description
FDwfDeviceCloseAll	Close all the open devices.

Figura 25 : Funções do Waveforms SDK

4.4.2 Detalhamento do Projeto de Software

Foram utilizadas principalmente as bibliotecas do Waveforms “WF_SDK.protocol” e “WF_SDK” e a biblioteca de integração do python “ctypes”. Das funções dadas pelo software para protocolo UART, as que se adequam melhor ao início dessa implementação foram as funções: *open, read e close* (descritas a seguir).

```python
def open(device_data, rx, tx, baud_rate=9600, parity=None, data_bits=8, stop_bits=1):
    """
        initializes UART communication

        parameters: - device data
                    - rx (DIO line used to receive data)
                    - tx (DIO line used to send data)
                    - baud_rate (communication speed, default is 9600 bits/s)
                    - parity possible: None (default), True means even, False means odd
                    - data_bits (default is 8)
                    - stop_bits (default is 1)
    """
    # set baud rate
    dwf.FDwfDigitalUartRateSet(device_data.handle, ctypes.c_double(baud_rate))

    # set communication channels
    dwf.FDwfDigitalUartTxSet(device_data.handle, ctypes.c_int(tx))
    dwf.FDwfDigitalUartRxSet(device_data.handle, ctypes.c_int(rx))

    # set data bit count
    dwf.FDwfDigitalUartBitsSet(device_data.handle, ctypes.c_int(data_bits))

    # set parity bit requirements
    if parity == True:
        parity = 2
    elif parity == False:
        parity = 1
    else:
        parity = 0
    dwf.FDwfDigitalUartParitySet(device_data.handle, ctypes.c_int(parity))

    # set stop bit count
    dwf.FDwfDigitalUartStopSet(device_data.handle, ctypes.c_double(stop_bits))

    # initialize channels with idle levels
```

```python
    # dummy read
    dummy_buffer = ctypes.create_string_buffer(0)
    dummy_buffer = ctypes.c_int(0)
    dummy_parity_flag = ctypes.c_int(0)
    dwf.FDwfDigitalUartRx(device_data.handle, dummy_buffer, ctypes.c_int(0), ctypes.byref(dummy_buffer), ctypes.byref(dummy_parity_flag))

    # dummy write
    dwf.FDwfDigitalUartTx(device_data.handle, dummy_buffer, ctypes.c_int(0))
    return
```

Figura 26 : Função open do Waveforms SDK

```python
def read(device_data):
    """
        receives data from UART

        parameters: - device data
        return:     - integer list containing the received bytes
                    - error message or empty string
    """
    # variable to store results
    error = ""
    rx_data = []

    # create empty string buffer
    data = (ctypes.c_ubyte * 8193)()

    # character counter
    count = ctypes.c_int(0)

    # parity flag
    parity_flag= ctypes.c_int(0)

    # read up to 8k characters
    dwf.FDwfDigitalUartRx(device_data.handle, data, ctypes.c_int(ctypes.sizeof(data)-1), ctypes.byref(count), ctypes.byref(parity_flag))

    # append current data chunks
    for index in range(0, count.value):
        rx_data.append(int(data[index]))

    # ensure data integrity
    while count.value > 0:
        # create empty string buffer
        data = (ctypes.c_ubyte * 8193)()

        # character counter
        count = ctypes.c_int(0)
```

```python
        # parity flag
        parity_flag= ctypes.c_int(0)

        # read up to 8k characters
        dwf.FDwfDigitalUartRx(device_data.handle, data, ctypes.c_int(ctypes.sizeof(data)-1), ctypes.byref(count), ctypes.byref(parity_flag))
        # append current data chunks
        for index in range(0, count.value):
            rx_data.append(int(data[index]))

        # check for not acknowledged
        if error == "":
            if parity_flag.value < 0:
                error = "Buffer overflow"
            elif parity_flag.value > 0:
                error = "Parity error: index {}".format(parity_flag.value)
    return rx_data,
```

Figura 27: Função read do Waveforms SDK

```python
def close(device_data):
    """
        reset the uart interface
    """
    dwf.FDwfDigitalUartReset(device_data.handle)
    return
```

Figura 28 : Função close do Waveforms SDK

4.5 Plano de Testes do Software

4.6 Integração entre Hardware Programável e Software[2]

4.7 Prototipação do Projeto

Nesta iteração do protótipo continuamos sem os outros componentes do sistema (computador e painel de botões), portanto ele consiste apenas na placa FPGA ainda.

4.8 Implantação do Projeto

4.8.1 Pinagem da placa FPGA

Node Name	Direction	Location	I/O Bank	VREF Group	I/O Standard	Reserved	urrent Streng	Slew Rate	ifferential Pai	Analog Settir	(B/VCCT_GXE
botoes[6]	Input	PIN_AA14	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)				
botoes[5]	Input	PIN_AB15	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)				
botoes[4]	Input	PIN_AA15	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)				
botoes[3]	Input	PIN_T12	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)				
botoes[2]	Input	PIN_T13	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)				
botoes[1]	Input	PIN_V13	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)				
botoes[0]	Input	PIN_U13	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)				
clock	Input	PIN_A13	7A	B7A_N0	2.5 V ...fault)		12mA ...ault)				
confirmar	Input	PIN_B12	7A	B7A_N0	2.5 V ...fault)		12mA ...ault)				
db_clock	Output	PIN_N1	2A	B2A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_confirmar	Output	PIN_L1	2A	B2A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_contagem_erros[6]	Output	PIN_P9	3B	B3B_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_contagem_erros[5]	Output	PIN_Y15	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_contagem_erros[4]	Output	PIN_U15	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_contagem_erros[3]	Output	PIN_U16	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_contagem_erros[2]	Output	PIN_V20	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_contagem_erros[1]	Output	PIN_Y20	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_contagem_erros[0]	Output	PIN_U20	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_endereco_letra[6]	Output	PIN_U22	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_endereco_letra[5]	Output	PIN_AA17	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_endereco_letra[4]	Output	PIN_AB18	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_endereco_letra[3]	Output	PIN_AA18	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_endereco_letra[2]	Output	PIN_AA19	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_endereco_letra[1]	Output	PIN_AB20	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_endereco_letra[0]	Output	PIN_AA20	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_estado[6]	Output	PIN_W19	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_estado[5]	Output	PIN_C2	2A	B2A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_estado[4]	Output	PIN_C1	2A	B2A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_estado[3]	Output	PIN_P14	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_estado[2]	Output	PIN_T14	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_estado[1]	Output	PIN_M8	3B	B3B_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_estado[0]	Output	PIN_N9	3B	B3B_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_estado_serial[6]	Output				2.5 V ...fault)		12mA ...ault)	1 (default)			
db_estado_serial[5]	Output				2.5 V ...fault)		12mA ...ault)	1 (default)			
db_estado_serial[4]	Output				2.5 V ...fault)		12mA ...ault)	1 (default)			
db_estado_serial[3]	Output				2.5 V ...fault)		12mA ...ault)	1 (default)			
db_estado_serial[2]	Output				2.5 V ...fault)		12mA ...ault)	1 (default)			
db_estado_serial[1]	Output				2.5 V ...fault)		12mA ...ault)	1 (default)			
db_estado_serial[0]	Output				2.5 V ...fault)		12mA ...ault)	1 (default)			
db_igual	Output	PIN_L2	2A	B2A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			
db_iniciar	Output	PIN_U1	2A	B2A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)			

Figura 29: Pin Planner do Intel Quartus Prime

[2] Como o software não estava completo, não foram elaborados plano de testes e a integração entre Hardware e Software também não foi possível devido a problemas na utilização concorrente de uma porta para dois programas diferentes: TeraTerm e Waveforms

db_letra_less[6]	Output	PIN_AB21	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_letra_less[5]	Output	PIN_AB22	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_letra_less[4]	Output	PIN_V14	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_letra_less[3]	Output	PIN_Y14	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_letra_less[2]	Output	PIN_AA10	3B	B3B_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_letra_less[1]	Output	PIN_AB17	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_letra_less[0]	Output	PIN_Y19	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_letra_more[6]	Output	PIN_V19	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_letra_more[5]	Output	PIN_V18	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_letra_more[4]	Output	PIN_U17	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_letra_more[3]	Output	PIN_V16	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_letra_more[2]	Output	PIN_Y17	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_letra_more[1]	Output	PIN_W16	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_letra_more[0]	Output	PIN_Y16	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_modo	Output	PIN_N2	2A	B2A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_rom_escolhida[6]	Output	PIN_AA22	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_rom_escolhida[5]	Output	PIN_Y21	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_rom_escolhida[4]	Output	PIN_Y22	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_rom_escolhida[3]	Output	PIN_W21	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_rom_escolhida[2]	Output	PIN_W22	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_rom_escolhida[1]	Output	PIN_V21	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_rom_escolhida[0]	Output	PIN_U21	4A	B4A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_saida_serial	Output	PIN_K16	7A	B7A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_tem_jogada	Output	PIN_U2	2A	B2A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
db_tick_serial	Output				2.5 V ...fault)		12mA ...ault)	1 (default)		
ganhou	Output	PIN_AA1	2A	B2A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
jogar	Input	PIN_H15	7A	B7A_N0	2.5 V ...fault)		12mA ...ault)			
leds[6]	Output	PIN_R15	5A	B5A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
leds[5]	Output	PIN_P18	5A	B5A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
leds[4]	Output	PIN_K19	7A	B7A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
leds[3]	Output	PIN_K17	5B	B5B_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
leds[2]	Output	PIN_L19	5B	B5B_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
leds[1]	Output	PIN_L17	5B	B5B_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
leds[0]	Output	PIN_L18	5B	B5B_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
modo	Input	PIN_A12	7A	B7A_N0	2.5 V ...fault)		12mA ...ault)			
perdeu	Output	PIN_AA2	2A	B2A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
pronto	Output	PIN_W2	2A	B2A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
pronto_serial	Output	PIN_K20	7A	B7A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		
reset	Input	PIN_H16	7A	B7A_N0	2.5 V ...fault)		12mA ...ault)			
saida_serial	Output	PIN_D17	7A	B7A_N0	2.5 V ...fault)		12mA ...ault)	1 (default)		

Figura 30: Pin Planner do Intel Quartus Prime (continuação)

4.8.2 Estratégia de Montagem

a) Programação da placa FPGA

Como usual, a placa FPGA será conectada ao computador do laboratório por meio de um cabo USB (servirá tanto como alimentação, quanto para transferência de dados), e a programação da placa será feita utilizando a ferramenta ¨Programmer¨ do Intel Quartus.

b) Ligações de terminais e fios

A conexão principal será realizada entre a FPGA e o computador da bancada utilizando um cabo USB. Ademais, para uso do dispositivo AnalogDiscovery, será necessário interligar os equipamentos com o uso de jumpers da seguinte forma:

GND: Pino 12

Clock: Pino 14

Reset: Pino 16

Botões (0 a 3): Pinos 22,24,26 e 28, respectivamente

Leds (0 a 3): Pinos 32,34, 36 e 38, respectivamente

Iniciar: Pino 18

Modo: Pino 20

Obs. 1: Sempre ligar primeiramente o terra (GND) e depois dar seguimento às outras conexões.

Obs. 2: A pinagem descrita acima se refere exclusivamente ao barramento GPIO–0.

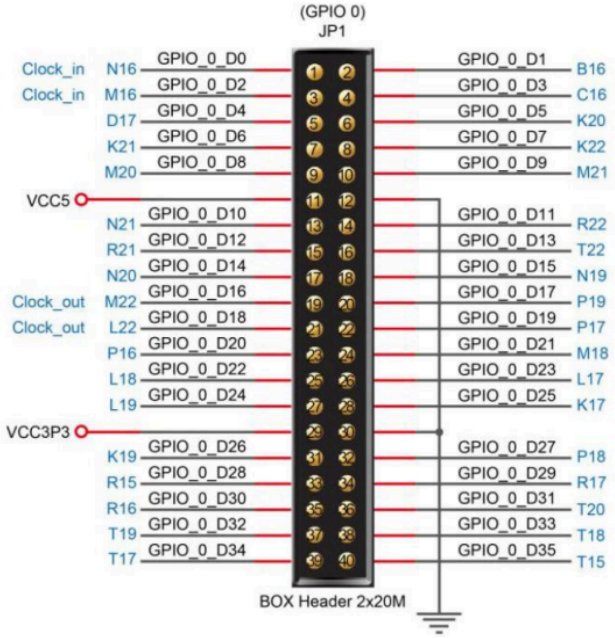

Figura 31: Ilustração dos conectores do GPIO-0 da placa DE0-CV com numeração

c) Sinais/elementos físicos monitorados

Os elementos físicos da placa que serão utilizados para interagir e monitorar o sistema são:

d) Entradas: Todos os pinos GPIO conectados no Analog Discovery através do software waveform.

e) Saídas binárias: LEDS 0 a 9

f) Saídas de dados: Displays de sete segmentos 0,1,2 e 5

d) Sinais de depuração considerados e como são monitorados

Os sinais considerados serão explicitados no item 3.8.3a do relatório e estes serão monitorados a partir dos leds e displays mencionados anteriormente (item 3.8.2.c).

4.8.3 Estratégia de Depuração

a) **Sinais de depuração:**

Foram incluídos os sinais de depuração da experiência anterior:

a) db_estado (Saída para HEX5): Exibe o estado atual da FSM. Fundamental para verificar se o sistema está transicionando de estados de maneira coerente.
b) db_contagem (Saída para HEX0): Monitora o primeiro endereço da memória escolhida pela aleatorização. Permite validar se a lógica com os registradores de palavras e de memória e o contador da palavra estão funcionando corretamente
c) db_memoria (Saída para HEX1): Exibe qual das memórias foi a escolhida, fundamental para verificação.
d) debug_dado (Saída para HEX2): Mostra o dado armazenado no endereço mostrado por db_contagem
e) db_contagem_erros (Saída para HEX3): Mostra o valor que foi efetivamente registrado no registrador_4. Permite verificar se a leitura do dado inserido pelas chaves foi correta.
f) db_tem_jogada, db_clock, db_igual, db_iniciar, db_confirmar (Saídas para LEDs): Indicadores de atividade (heartbeat). O db_tem_jogada confirma se o sinal elétrico das chaves está chegando ao edge detector, enquanto o db_clock confirma a integridade do sinal de clock externo e db_igual indica se o valor das chaves é igual ao da memória. Por último, db_confirmar mostra o pulso gerado pelo *edge_detector* ao apertar o botão de confirmar.
g) db_modo (saída para leds): db_modo indica o modo escolhido. Importante para verificar se há efeito no circuito ao alterar o modo durante a partida (o que não deveria ocorrer).

b) **Monitoramento dos sinais**

Os sinais serão monitorados visualmente a partir dos leds e displays de 7 segmentos da placa FPGA.

c) **Possíveis alterações no projeto lógico**

Caso necessário, as alterações serão feitas nos códigos verilog dos componentes

utilizados e, com isso, o circuito será novamente compilado e sintetizado para, enfim, reprogramar a placa.

d) **Possíveis alterações na montagem**

Se necessário, pode-se alterar a pinagem dos componentes da placa, sempre em conformidade com a tabela de pinos para a placa DE0-CV disponibilizada na página da disciplina no e-disciplinas.

e) Testes de regressão

Para qualquer alteração feita, seja na lógica do circuito ou na montagem, todos os testes serão reexecutados, podendo ainda adicionar mais casos de testes, para garantir o correto funcionamento do circuito em todos os aspectos.

4.8.5 Execução Prática do Cenário de Teste 1 - Vitória e permanência no estado

Caso de teste para validar a capacidade do circuito de ler a memória escolhida na sequência correta e comparar corretamente cada entrada do usuário com a letra correspondente, parando ao fim da palavra e terminando em vitória do usuário. Estando nesse estado terminal, também será validado que o circuito não lerá mais nenhuma jogada do usuário até que iniciar ou reset sejam pressionados e o sistema volta ao estado inicial.

Obs.: Como os valores dependerão da memória escolhida nos testes práticos, tanto nesse cenário quanto nos seguintes, a tabela que irá descrevê-los será preenchida após a execução e colocada junto com um vídeo mostrando o funcionamento do circuito.

Tabela 20: Descrição e resultados do cenário de testes práticos 1 da semana 2

Teste	Sinais de entrada			confirmar	Resultado Esperado	Resultado Observado
	reset	iniciar	chaves			
C.I	0	0	0000	0	-	Ok
1	1	-	-	-	-	Ok
2	0	-	-	-	-	Ok
3	-	1	-	-	saída db_iniciar ativada	Ok

4	-	0	0010	1	seletor_rom_out = 10 s_endereco = 1000 s_letra = 0010	Ok
5	-	-	0111	1	seletor_rom_out = 10 s_endereco = 1001 s_letra = 0111	Ok
6	-	-	1110	1	seletor_rom_out = 10 s_endereco = 1010 s_letra = 1110	Ok
7		-	-	-	seletor_rom_out = 10 s_endereco = 1011 s_letra = 0000 pronto = 1 acertou = 1	Ok

Como é possível observar no vídeo gravado da execução deste cenário 20260323_103601.mp4, é possível validar o correto funcionamento da lógica principal do jogo, encerrando sua execução no estado terminal de vitória, caso a sequência correta de jogadas seja feita pelo usuário.

4.8.6 Execução Prática do Cenário de Teste 2 - Validação da lógica de derrota

Busca validar o funcionamento da lógica prioritária do sinal reset em qualquer estado de funcionamento do circuito.

Tabela 21: Descrição e resultados do cenário de testes práticos 1 da semana 2

Teste	Sinais de entrada			confirmar	Resultado Esperado	Resultado Observado
	reset	iniciar	chaves			
C.I	0	0	0000	0	-	Ok
1	1	-	-	-	-	Ok
2	0	-	-	-	-	Ok

3	-	1	-	-	saída db_iniciar ativada	Ok
4	-	0	0010	1	seletor_rom_out = 00 s_endereco = 0000 s_letra = 0011 db_contagem_erros = 0	Ok
5	-	-	0111	1	seletor_rom_out = 00 s_endereco = 0000 s_letra = 0011 db_contagem_erros = 1	Ok
6	-	-	1110	1	seletor_rom_out = 00 s_endereco = 0000 s_letra = 0011 db_contagem_erros = 2	Ok
7		-	-	-	seletor_rom_out = 00 s_endereco = 0000 s_letra = 0011 pronto = 1 errou = 1	Ok

O vídeo 20260323_103722.mp4 demonstra que a placa fpga apresenta comportamento fielmente compatível com o esperado. Portanto, a lógica de limitação de erros por partida está validada.

4.9 Conclusão da Semana

Até o planejamento desta semana partes importantes da arquitetura da FPGA foram finalizadas, com isso, durante a aula prática, após rápida validação dos novos requisitos funcionais, o grupo concentrará seus esforços na integração entre placa e computador por meio da comunicação serial, para que, idealmente, já possível validar essa parte na semana 3 e dar seguimento a criação de uma interface gráfica e o painel de botões.

5 Conclusões do Projeto

Ao fim das aulas extensionistas (AEX 1 a 4), além das semanas 0.1 e 0.2, foi possível adquirir uma bagagem teórica extremamente importante para identificar

problemas e buscar eventuais soluções voltadas à comunidade TEA. Com isso foi definido que o objetivo do projeto desenvolvido nesta disciplina será o auxílio à alfabetização de crianças no espectro autista, uma vez que isso representa um aspecto fundamental na busca desse público por independência e grande melhoria de qualidade de vida.

Para implementá-lo, o projeto contará com a adoção de uma arquitetura modular, que divide as responsabilidades do hardware tangível (FPGA e Painel) com a interface audiovisual do software (Computador), o que demonstrou ser a solução mais robusta e factível para o funcionamento correto desse projeto.

Além disso, diversas ideias foram discutidas e aprimoradas durante o desenvolvimento do projeto no horário do laboratório, permitindo com que um planejamento do projeto semana a semana esteja melhor concretizado para os integrantes.

Na semana 1, o grupo deu pequenos passos em direção a compreensão das funcionalidades do projeto e consolidação do cronograma, concentrando esforços, portanto, em estruturar o desenvolvimento do trabalho e modificar circuitos de experiências anteriores. Como perspectiva de trabalhos futuros e próximos passos para as semanas seguintes, o grupo dará início à implementação da comunicação UART entre a FPGA e o computador e planeja finalizar a lógica do circuito interno da FPGA durante a segunda semana. Já na terceira semana, é esperado que sejam realizados poucos ajustes na comunicação serial e que se inicie tanto a implementação da interface gráfica quanto a montagem física do painel. Por último, na quarta semana espera-se que todas as etapas: circuito interno da FPGA, comunicação serial, painel físico, interface gráfica e eventuais elementos de software estejam em processo de ajustes finais e melhorias.

A expectativa é que, com este planejamento delineado, o protótipo do *SomLetrando* chegue à Feira de Projetos como uma ferramenta assistiva funcional, validada e com elevado potencial de impacto social.

6 Referências Bibliográficas

[1] LAZ, L. ScholarWorks Teaching Emergent Literacy Skills to Students with Autism. [s.l: s.n.]. Disponível em: <https://scholarworks.boisestate.edu/cgi/viewcontent.cgi?article=1003&context=sped_gradproj>.

[2] NUNES, D. R. DE P.; WALTER, E. C. Processos de Leitura em Educandos com Autismo: um Estudo de Revisão. **Revista Brasileira de Educação Especial**, v. 22, n. 4, p. 619–632, dez. 2016.

[3] **Nonverbal Reading**. Disponível em: <https://www.eiltsfamily.org/udl_at/resources/Reading/Nonverbal_reading.pdf>. Acesso em: 09 mar. 2026.

[4] Spector, J.E. Sight Word Instruction for Students with Autism: An Evaluation of the Evidence Base. *J Autism Dev Disord* **41**, 1411–1422 (2011). https://doi.org/10.1007/s10803-010-1165-x

[5] Vista do NEUROAPRENDIZAGEM E ALFABETIZAÇÃO DE CRIANÇAS COM TRANSTORNO DO ESPECTRO DO AUTISMO: COMO ENSINAR AOS NÍVEIS 1, 2 E 3 DE SUPORTE NO 1o ANO DO ENSINO FUNDAMENTAL. Disponível em: <https://realizeducscientificjournal.com/index.php/rsj/pt_BR/article/view/5/4>. Acesso em: 9

mar. 2026.

[6]

[7] OBERG, R.; PROBASCO, L.; ERICSSON, M.; *Applying Requirements Management with Use Cases*. Rational Software Corporation. Technical Paper TP505 (Version 1.4), 2001.

[8] BRITES, L. Métodos de alfabetização: qual o mais adequado às crianças autistas? - Instituto NeuroSaber. Disponível em: <https://institutoneurosaber.com.br/artigos/metodos-de-alfabetizacao-qual-o-mais-adequado-criancas-autistas/>.

[9] ALMEIDA, Izabel Cristina Araujo. Alfabetização de alunos com transtorno do espectro autista (TEA) : concepções e práticas dos professores. 2019. 147 f. Dissertação (Mestrado Acadêmico em Educação) - Universidade Estadual de Feira de Santana, Feira de Santana, 2019.

[10] BRITES, L. Consciência Fonológica em Crianças com Autismo - Instituto NeuroSaber. Disponível em: <https://institutoneurosaber.com.br/artigos/consciencia-fonologica-no-tea/>.